Contents

1 Introduction

The program Polyray is a rendering program for producing scenes of 3D shapes and surfaces. The means of description range from standard primitives like box, sphere, etc. to 3 variable polynomial expression, and finally (and slowest of all) surfaces containing transcendental functions like sin, cos, log. The files associated with Polyray are distributed in four pieces: the executable, a collection of document files, a collection of sample data files, and a few small utility programs. This open source version is now maintained and modified by myself Dr Clyde Meli.

Version 1.5 to 1.8 of Polyray were Shareware. Further versions are modified versions starting from the source code of 1.8 with bug fixes and alterations.

1.1 Historical Note

Please note that the new modified versions are not supported by the original author, but I still encourage you to support the original author for his development by sending $35 to

```
Alexander Enzmann
20 Clinton St.
Woburn, Ma 01801
USA
```

Please include your name and mailing address.

The original author wrote: “If you formally register this program, you achieve a state of mental bliss.
In addition you will be contributing to my ability to purchase software tools to make Polyray a better program. If you don’t register this program, don’t feel bad - I’m poor too - but you also

shouldn’t expect as prompt a response to questions or bugs. Consider it guilt-free Shareware. (I’m no longer distributing updates, the mailing costs were getting prohibitive and net access is getting cheap.) Note that all of the sample files in PLYDAT are Public Domain, you may use them freely.

PLEASE DO NOT BOTHER ALEXANDER ENZMANN WITH BUG REPORTS ON THIS POLYRAY VERSION! IT IS NOT SUPPORTED BY HIM. Current emails for Alexander Enzmann are unknown. Old emails were:

Compuserve as: Alexander Enzmann 70323,2461

Internet as: xander@mitre.org

Many thanks go to David Mason for his numerous comments and suggestions (and for adding a new feature to DTA every time I needed to test something).
Thanks also to the Cafe Pamplona in Cambridge Ma. for providing a place to get heavily caffinated and rap about ray-tracing and image processing for hours at a time.

Many thanks to the folks that let me include some of their scenes in the data archive: Jeff Bowermaster, Dan Farmer, and Will Wagner.

2

1.1.2 Modern Note

I have been updating Polyray especially for use at the University of Malta in the Applied Computer Graphics unit. Since version 1.9.4 which is a maintenance release, Polyray is now released as open source under the MIT License with the blessing of the original author Alexander Enzmann. Allow me to express a BIG Thank you.

The data files are ASCII text, the output image file format supported is Targa. Input images (for texturing, height fields, etc.) may be: Targa (all variants defined in the Truevision spec), PNG (all variants should work), or JPEG (JFIF format). The Targa format is supported by many image processing programs so if you need to translate between Targa and something else it is quite simple. The utility CJPEG, which converts from Targa to JPEG, is not included in the utility archive - indeed you can use any graphics program like Irfanview to convert graphics files. Polyray is case sensitive, so the following ways of writing foo are all considered

different: Foo, FOO, and foo. For an abbreviated list of Polyray's syntax, see the file quickref.txt.

Polyray works in console mode under Windows. I am working on developing a new TUI which would be a practical modern interface. Work is ongoing on a newer version of Polyray written in C++ still compatible with existing scene files.

1.1.3 License, Acknowledgments and Other Legal Stuff

This software is based in part on the work of the Independent JPEG Group.

Current Contact: Please visit the github: https://github.com/cwebdesign/polyray19pub

1.2 Useful tools

There are several types of tools that you will want to have to work with this program (in relative order of importance):

- An ASCII text editor for preparing and modifying input files.
- A picture viewer for viewing images.

- An image processing program for translation between Targa and some other format.
- An animation generator that will take a series of Targa images and produce an animation file.
- A media player for playing an animation on the screen eg Media Player Classic HC.

There are a number of tools out there that can be used in conjunction with Polyray. These have been somewhat arbitrarily divided them into categories, with no particular order of importance. Most of the following should be available online.

1.2.1 Animation Utilities

Many of the features in Polyray are specifically to support the generation of multiple frames. Once the frames are generated, additional tools are necessary to format and play the resulting animation. Top picks in this area are:

- Virtualdub (Avery Lee)
- DTA (David Mason)

Dave's Targa Animator (DTA) may be the only tool you need for creating FLI/FLCs from images, converting images between formats, compositing images, etc. For building animations with Polyray, all you need is DTA and a FLI/FLC player - you can use Media Player Classic or even Windows Media Player for that. The last version of DTA 3.0 is available at: https://www.povray.org/ftp/pub/povray/utilities/dta/

Using Virtualdub you can combine multiple images into a single AVI file, which afterwards can be converted into a Matroska mkv file with Mkvtoolnix.

Other programs of note include: FFmpeg, Blender, ImageMagick, Meshlab, GIMP/Krita, Davinci Resolve/Powerdirector (for video editing).

1.2.2 Modellers

There are a few really good modellers out there. Each of them allow the creation of objects in native form (rather than as hundreds of polygons), and provide fast, interactive, development of scenes using wireframe displays.
The modellers are:

• Moray (Lutz Kretzschmar)
• GUM (Alexander Van Der Sluijs)
• POV-CAD (Alphonso Hermida)

Moray is a graphical modeller for both Polyray and POV-Ray. Moray currently supports a large number of Polyray primitives directly, and provides a means for user defined objects and textures within a final data file. Recent versions have dropped support for Polyray. The latest version is found at https://www.povray.org/ftp/pub/moray/v3.5/

GUM, written by Alexander Van Der Sluijsis a graphical modeling program for Windows. It supports rendering with both Polyray and POV-Ray. This appears to be abandonware.

POVCAD, written by Alfonso Hermida is a graphical modeling program with both Windows and DOS versions. POV-CAD allows you to quickly build models through point and click in a wireframe environment. Both Polyray and POV-Ray data files can be generated.

I am planning an OBJ to Polyray converter, since most modern modellers export in OBJ format.

1.2.3 Miscellaneous Programs

• CTDS (Truman Brown)
• RAW2POV (Steve Anger)
• 3DS2POV (Steve Anger)
• SPD (Eric Haines, updates by Eduard Schwan & Alexander Enzmann)

CTDS is dot-to-dot for renderers. By creating a file listing lots of points and associated radii, this program will connect them together using spheres and cones. It can output to the native input of several renderers, including: Polyray, POV-Ray, and Vivid. This programs really fun for creating abstract shapes. You can think of it as a sphere sweep.

RAW2POV is a conversion program that takes ASCII files of triangle vertices and creates an input for various renderers. It can also examine the triangles and calculate normals for the vertices, giving a very smooth looking object when rendered. This is a must have utility if you are going to be converting polygonal objects into a form that can be used by Polyray. It is available in the tools directory in the Polyray github.

3DS2POV converts model files built by the costly 3DS modeller/ renderer into the native input of various renderers. Really nice to have if you are getting models in 3DS format or if you are fortunate to have 3DS yourself. It is available in the tools directory in the Polyray github.

SPD is a library of C code originally developed to benchmark various renderers. Similar in nature to OORT (described below), it allows you to write short C programs that will generate scene files for a number of different renderers. Currently supported are: Polyray, POV-Ray, Vivid, Rayshade, Rend386, RTrace, NFF, and QRT, DXF, RIB, RAW, Art, and 3DMF. There are also routines for displaying the models in wireframe (have to be tuned for individual compilers and platforms). New output modes and rendering formats are still being added. It is available at: https://www.realtimerendering.com/resources/SPD/

1.2.4 Other Raytracers

If you want to explore the world of raytracers, below is a list of what I consider the best of the Shareware/Freeware world. (No flame wars please, these are just ones I happen to have and like.) Each has it’s own strengths and weaknesses. Overall, you will learn something from working with each one of them.

- POV-Ray (The POV-Ray Team)
- Vivid (Stephen Coy)
- Rayshade (Craig Kolb)
- Raggier (C. Meli)

POV-Ray is the camel that you let warm it’s nose in your tent. It’s starting to really dominate the scene. It’s strengths are in a broad set of features together with a huge level of support. The authors are available on Compuserve (Graphdev) to answer questions, there are an astonishing number of utilities, and the source code is available for porting it to different platforms.

For a really fast raytracer, Vivid can’t be beat. It’s DOS based only, but the registered version comes with a DOS extender for those really huge scene files. Shape primitives are somewhat limited, but the texturing capabilities and camera lens features are very strong.

Rayshade is an old standard for the UNIX workstation crowd. It has just about all of the shapes you would want, texturing approximately

on a par with POV- Ray, and an interesting gridded optimization technique. It doesn't do graphics, and it's really slow if you don't manually tune the optimization. On the plus side it has a long background of people using it and will compile on just about anything (source code is available). Version 4 is available on github.

1.2.5 References

Over the last year or so there have been several books published that describe ways to use Polyray. Although these books were written about v1.6a, almost all of the content is still valid for v1.9. The differences are pretty well described in section 5. Other differences are buried here and there in this document.

Of the books published, the three that really dig into using Polyray for various tasks, from modeling to animations are:

Making Movies on Your PC David Mason & Alexander Enzmann Waite Group Press, 1993 ISBN 1-878739-41-7

Adventures in Ray Tracing Alphonso Hermida Que, 1993 ISBN 1-56529-555-2

Animation How-To CD Jeff Bowermaster Waite Group Press, 1994 ISBN 1-878739-54-9

There's good stuff in each of these books, so it's not unreasonable to actually get all of them. (I don't get royalties from any of them, so this is a reasonably shameless plug.)

1.3 Contents of this document

This document describes the input format and capabilities of the Polyray ray- tracing program. The following features are supported:

- Viewpoint (camera) characteristics
- Lights: point, directional, spotlight, functional, and area
- Background color
- Shape primitives: Bezier patch, blob, box, cone, cylinder, disc, glyph, implicit function, height field, hypertexture, lathe surface, NURBS, parabola, parametric function, polygon, polynomial function, sphere, superquadric, sweep surface, torus, triangular patches
- Animation support
- Conditional processing

- Include files
- Named values and objects
- Constructive Solid Geometry (CSG)
- Grids of objects
- User definable (functional) textures
- Initialization file for default values

1.4 Quick demo

This section describes one of the simplest possible data files: a single sphere and a single light source. In what follows anything following a double slash, // is a comment (similar to C++) and is ignored by Polyray. The data file is the following indented lines. You can either use the file sphere.pi in the data archive, or copy these declarations into a new file.

```
// We start by setting up the camera.  This is where the eye is
// located, and describes how wide a field of view will be used,
// and the resolution (# of pixels wide and high) of the resulting
// image.
viewpoint {
   from <0,0,-8>        // The location of the eye
   at <0,0,0>           // The point that we are looking at
   up <0,1,0>           // The direction that will be up
   angle 45             // The vertical field of view
   resolution 160, 160  // The image will be 160x160 pixels
   }

// Next define the color of the background.  This is the color
// that is used for any pixel that isn't part of an object.  In
// this file it will color the image around the sphere.
background skyblue

// Next is a light source.  This is a point light source a little
// above, to the left, and behind the eye.
light <-10,3, -20>

// The following declaration is a texture that will be applied
// to the sphere.  It will be red at every point but where the
// light is reflecting directly towards the eye - at that point
// there will be a bright white spot.
define shiny_red
texture {
   surface {
      color red
      ambient 0.2         // Always a little red in the sphere
      diffuse 0.6         // Where the light is hitting the
                          // sphere, the color of the sphere
```

```
                        // will be a little brighter.
      specular white, 0.5 // There will be a white highlight.
      microfacet Cook 5   // The white highlight drops to half
                          // its intensity at an angle of 5
                          // degrees.
      }
   }

// Finally we declare the sphere.  It sits right at the origin,
// and has a radius of two.  Following the declaration of the
// sphere, we associate the texture declared above.
object {
   sphere <0, 0, 0>, 2
   shiny_red
   }
```

Now that we have a data file, lets render it and show it on the screen. First of all we can look at a wireframe representation of the file. Enter the following commands:

```
polyray sphere.pi -r 2 -V 1 -W
```

An outline of the sphere will be drawn on the screen, press any key to get back to DOS. Next lets do a raytrace. Enter the following:

```
polyray sphere.pi -V 1
```

The sphere will be drawn a line at a time, when it is done you will be returned to the shell.

The output image will be the default output file, out.tga. You can view it directly in windows by double clicking it but you have to install an application such as ImageGlass, IrfanView, or XnView.

Now that you have seen a simple image, you have two options: go render some or all of the files in the data archive, PLYDAT, or continue reading the documents. For those of you that prefer immediately getting started, there are a series of DOS batch files that pretty well automate the rendering of the sample scenes. (When you unzip the data files, remember to use the -d switch so that all the subdirectories will be installed properly.) The sample scenes in PLYDAT will render in a few minutes.

1.5 Command Line Options

A number of operations can be manipulated through values in an initialization file, within the data file, or from the command line (processed in that order, with the command line flags having the

highest precedence). The command line values will be displayed if no arguments are given to Polyray.

The values that can be specified at the command line, with a brief description of their meaning are:

Status options:

```
-t status_vals  Status display type [0=none,1=totals,2=line,3=pixel].
```

Antialiasing options for ray-tracing:

```
  -a mode         Antialiasing (0=none,1=corner average,2-4=adaptive)
  -S samples      # of samples per pixel when performing focal blur
  -T threshold    Threshold to start oversampling (default 0.2)
```

Optimization options:

```
   -O optimizer    0 = none, 1 = slabs
```

Display option:

```
   -V mode         Display mode while tracing
                       [    0=none,    1-5=8bit,
6-10=15bit,
                        11-15=16bit, 16-20=24bit, 21-22=4bit/EGA  ]
   -P palette     Which palette to use [0=grey, 1=666, 2=884, 3=EGA]
   -e start        Starting index of palette in VGA palette
   -W              Wait for key before clearing display
   -D dither       Dithering of display [0=no dither, 1 = use dither]
```

Frame counter option:

```
   -F start_frame  Skip frames until start_frame
```

Abort option:

```
   -Q abort_option 0=no abort, 1=check by pixel, 2=by line/object
```

File options:

```
   -R                      Resume an interrupted trace
   -z y0 y1                Render from line y0 to line y1
   -u                      Write the output file in uncompressed form
   -x columns              Set the x resolution
   -y lines                Set the y resolution
   -z start_line,fin_line  Start a trace at a specified line to another line
   -o filename             Output file name (default "out.tga")
   -p bits/pixel           Number of bits per pixel 8/16/24/32 (default 24)
   -d                      Render as a depth map
   -N                      Don't generate an image file
```

Rendering options:

```
-M kbytes       Max # of KBytes for image buffer
-q flags        Turn on/off various global shading options
                  [1=shadows, 2=reflect, 4=transmit,
                   8=two sides, 16=check uv, 32=correct normals,
                   63=all flags]
-r renderer     Rendering method: [0=raytrace, 1=scan convert,
                2=wireframe, 3=hidden line, 4=Gouraud shaded,
                5=raw triangle information,6=uv triangles,
                7=csg_triangles]
-G              Output in PPM format (unsupported at the moment)
```

If no arguments are given then Polyray will give a brief description of the command line options.

1.6 Initialization File

The first operation carried out by Polyray is to read the initialization file polyray.ini. This file can be used to tune a number of the default variables used during rendering. Polyray searches for polyray.ini in all directories that appear in the environment variable POLYRAY_PATH. This variable works just like the PATH variable does in Windows.

The initialization file doesn’t have to exist, it is typically used as a convenience to eliminate retyping command line parameters.

Each entry in the initialization file must appear on a separate line, and have the form:

```
default_name default_value
```

The names are text. The values are numeric for some names, and text for others. The allowed names and values are:

Name	Allowed value(s)	Meaning
abort_test	true / false / on / off / slow	Controls abort testing behaviour
alias_threshold	numeric value	Value that causes adaptive antialiasing to start
antialias	none / filter / adaptive1 / adaptive2	Selects the antialiasing mode
clustersize	numeric value	Number of objects stored in a slab
csg_tolerance		[Smallest size triangle that is checked for CSG]
csg_subdivisions		[Max # of subdivisions on CSG triangles]
display		none/vga1…vga5/hicolor1…hicolor5/
		16bit1…16bit5/truecolor1…truecolor5
dither	on/off	[Dither colors on the display]
error_log		[Name of file to write errors and warnings]
errors		on/off
max_level		[max depth of recursion]
max_samples		[# of samples to use with antialiasing, or focal blur]
maxscreenbuffer		[Max number of kbytes allocated to screen buffer]

Name	Allowed value(s)	Meaning
optimizer		none/bounding_slabs/bsp_tree
pallette		884/666/grey/4bit
pallette_start		[Start location the VGA palette]
pixel_size		8/16/24/32
pixel_encoding		none/rle
renderer		ray_trace/scan_convert/ wire_frame/ raw_triangles/uv_triangles
resolution		[xres, yres - default image size]
screen_window		[x0, y0, xl, yl - placement, size of preview image]
shade_flags		[default/bit mask of flags, see section 1.7.1.4]
shadow_tolerance		[miminum distance for blocking objects]
status		none/totals/line/pixel
warnings		on/off

Any lines starting with "//" will be treated as comments & ignored.

A typical example of `polyray.ini` would be:

Name	Example value
abort_test	on
alias_threshold	0.05
antialias	adaptive
display	vga
max_samples	8
pixel_size	24
status	line

If no initialization file exists, then Polyray will use the following default values:

Name	Default value
abort_test	on
alias_threshold	0.2
antialias	none
display	none
max_level	5
max_samples	4
optimizer	slabs
pixel_size	16
pixel_encoding	rle
renderer	ray_trace
shade_flags	default
shadow_tolerance	0.001
status	none
warnings	on

1.6.1 Description of initialization file

• Screen subwindow & Negative Video Modes

```
screen_window x0, y0, xl, yl
```

If you set the screen window, then all pixels in the image will be scaled to fit into the defined window. The first two values define the upper left corner of the window, the next two values define the width and height of the window. Note that if you have an image with a resolution lower than the size of the window, Polyray will scale the pixels larger so that the image fits the window. (Uniform scaling only, so it may not fit in both directions.)

This is particularly useful in conjunction with negative video mode flags if you want to incorporate Polyray displays into the displays of another program. If you have a program that runs in a supported Polyray video mode, then you can start Polyray with a system call plus a command line flag like: “-V –6” which would tell Polyray

that you are already in a 320x200 hicolor mode and you want to stay there.

• Default image resolution

```
resolution xres yres
```

If the default resolution is set in polyray.ini then all images that don’t have a resolution statement (or an overriding value on the command line) will use this resolution. Without this statement the default resolution for image is 256x256

• Maximum memory devoted to image buffering

```
maxscreenbuffer kbytes
```

This value limits the amount of memory used for the Z-buffer and image buffer.
The value given is in kilobytes. There must be enough for at least 3 image lines (Polyray will override to get at least this much). For example, the following entry in polyray.ini would limit the buffer storage to 256K bytes:

```
maxscreenbuffer 256
```

This value can also be set from the command line with the -M flag:

```
polyray foo.pi -M 256
```

• Error/Warning log file

Instead of having warning and error messages spit out to the screen, it is possible to have Polyray automatically send them to a log file. To do this you add a line like the following to polyray.ini:

```
error_log err.log
```

All warnings and errors will now be sent to “err.log”. If you are running Polyray from within another program, then you can check the return value (non- zero implies a problem) and print any warnings or errors that may appear in the log file.

• Abort test

If abort_test is set to “on” or “true”, then Polyray will check for a keypress after drawing every pixel. If one was hit, then rendering will be stopped and the value of the key will be returned (useful when running Polyray from another process).

If abort_test is set to “off” or “false”, then Polyray will never check for a keypress. This saves processing time, but if you need to stop the render you may have to reboot (or terminate the DOS box if you are in Windows).

```
abort_test slow
```

If you use “abort_test slow”, then Polyray will only check for a keypress to abort the render after an entire line in raytracing mode and after an entire object in scan/wireframe/raw mode.

The -Q flag from the command line can be used to change the way Polyray will abort. If the value after the Q is 0, then no abort test will be performed (except between frames, where an will cause an abort). If it is 1 then it’s the equivalent of “abort_test slow”. If 2 then an abort test is performed for every pixel drawn.

• Palette start

```
palette_start entry
```

Tells Polyray that if a palette based display mode is being used then the color entries should start at “entry”. No other entries in the palette should be modified. Note that the value of entry is limited by the total numbers of colors in the mode.

• csg_tolerance, csg_subdivisions

See the section on rendering modes for info on how these work

1.7 Rendering Options

Polyray supports four very distinct means of rendering scenes: raytracing, polygon scan conversion, wireframe, and raw triangle output. Raytracing is often a very time consuming process, however the highest quality of images can be produced this way. Scan conversion is a very memory intensive method, but produces a good quality image quickly. Wireframe gives a very rough view of the scene in the fastest possible time. (Note that there is no output file when wireframe is used.) Raw triangle output produces an ASCII file of triangles describing the scene.

Three new rendering modes are now supported: hidden line, Gouraud shading, and CSG Triangles. The rendering mode numbers have also changed. The list is now:

0. Raytracing

1. Scan line polygon rendering (full texturing)
2. Wire frame
3. Hidden line rendering
4. Gouraud shading (texturing at vertices only)
5. raw triangles
6. u/v triangles
7. CSG triangles

For example, to render a data file using hidden line you could type:

```
polyray sphere.pi -r 3
```

Hidden line is pretty much self explanatory - lines in the wireframe that aren't visible due to blocking by other polygons aren't shown. Gouraud shading performs a texturing calculation at the corners of all triangles and fills in color by interpolating between the vertices.

Another change is that modes 0 - 4 all generate an output image file. Previously the wire frame mode did not generate an image file.

1.7.1.1 Antialiasing

The representation of rays is as a 1 dimensional line. On the other hand pixels and objects have a definite size. This can lead to a problem known as aliasing, where a ray may not intersect an object because the object only partially overlaps a pixel, or the pixel should have color contributed by several objects that overlap it, none of which completely fills the pixel. Aliasing often shows up as a staircase effect on the edges of objects.

Polyray offers a couple of ways to reduce aliasing: by filtering or by adaptive oversampling. Filtering smoothes the output image by averaging the color values of neighboring pixels to the pixel being rendered. Oversampling is performed by adding extra rays around the original one. By averaging the result of all of the rays that are shot through a single pixel, aliasing problems can be greatly reduced.

The filtering process adds little overhead to the rendering process, however the resolution of the image is degraded by the averaging process. It simply averages the colors found at the four corners of a pixel.

Adaptive antialiasing starts by sending a ray through the four corners of a pixel. If there is a sufficient contrast between the corners then Polyray will fire five more rays within the pixel. This results in four subpixels.
Each level of adaptive antialiasing repeats this procedure. If adaptive1 is used in polyray.ini, or -a 2 from the command line, then the antialiasing stops after a single subdivision. If adaptive2 is used then each subpixel is checked and if the corners are sufficiently different then it will be divided again.

The two initialization (and command line) variables that will affect the performance of adaptive antialiasing are alias_threshold and max_samples. The first is a measure of how different a pixel must be with respect to its neighbors before antialiasing will kick in. If a pixel has the value: $<r_1, g_1, b_1>$, and it's neighbor has the value $<r_2, g_2, b_2>$ (RGB values between 0 and 1 inclusive), then the distance between the two colors is:

$$dist = sqrt((r_1 - r_2)^2 + (b_1 - b_2)^2 + (g_1 - g_2)^2)$$

This is the standard Pythagorean formula for values specified in RGB. If dist is greater than the value of alias_threshold, then extra rays will be fired through the pixel in order to determine an averaged color for that pixel.

Antialiasing is on by default.

1.7.1.2 Focal Blur

The default viewpoint declaration is a pinhole camera model. All objects are in sharp focus, no matter how near or far they are. By adjusting the aperture and focal_distance parameters it is possible to simulate a real world camera.
Objects close to the focal distance will be in focus and those closer or farther will be blurred. Three parameters affect the blur: aperture, focal_distance, and max_samples.

Adjusting the size of aperture allows you to adjust how much of the scene is in focus. Adjusting focal_distance gives you control over what part of the scene is in focus. If focal_distance isn't set, Polyray will set it so that the point defined by the 'at' declaration is in focus. The declaration max_samples allows you to fine tune how smooth the final image appears. If you use a large

value for aperture, you will probably need to increase max_samples as well to avoid a grainy appearance in the image.

Also note that focal blur and adaptive antialiasing interact with each other.
Polyray attempts to distribute the focal blur rays into the antialiasing rays, however there will be more rays cast than would have been using either method alone. It's best to leave the antialiasing off and the value of max_samples small until producing the final image.

1.7.1.3 Bounding Slabs

For scenes with large numbers of small objects, there are optimization tricks that can take advantage of the distribution of the objects. An easy one to implement, and one that often results in huge time savings are bounding slabs. The basic idea of a bounding slab is to sort all objects along a particular direction. Once this sorting step has been performed, then during rendering the ray can be quickly checked against a slab (which can represent many objects rather than just one) to see if intersection tests need to be made on the contents of the slab.

The most important caveats with Polyray's implementation of bounding slabs are:

- Scenes with only a few large objects will derive little speed benefits.
- If there is a lot of overlap of the positions of the objects in the scene, then there will be little advantage to the slabs.
- If the direction of the slabs does not correspond to a direction that easily sorts objects, then there will be little speed gained.

However, for data files that are generated by another program, the requirements for effective use of bounding slabs are often met. For example, most of the data files generated by the SPD programs will be rendered orders of magnitude faster with bounding slabs than without.

Slabs are turned on by default. To turn them off either use the command line flag, -O 0, or add the line, optimizer none, to polyray.ini.

1.7.1.4 Shading Quality Flags

By specifying a series of bits, various shading options can be turned on and off. The value of each flag, as well as the meaning are:

```
  1   shadow_check      Shadows will be generated
  2   reflect_check     Reflectivity will be tested
  4   transmit_check    Check for refraction
  8   two_sides         If on, highlighting will be performed
                        on both sides of a surface (based on normal).
 16   uv_check          Calculate the uv-coordinates of each point
 32   normal correct    Flip normals to point towards light
 64   cast_shadow       Determines if an object can cast a shadow
```

The default settings of these flags are:

Name	Default value
raytracing	Shadow_Check + Reflect_Check + Transmit_Check + UV_Check + Normal_Correct (= 23)
scan conversion	Two_Sides (= 8)
wireframe	Not applicable
raw triangles	Not applicable

Note that the flag, cast_shadow, is only meaningful for declarations of shading flags for an object, not for the renderer. See section 2.3.1.4 for more information. If you want to turn off shadows in a raytrace, then you would leave out the shadow_check flag (conversely if you wanted shadows in scan conversion you would add it).

The assumption made is that during raytracing the highest quality is desired, and consequently every shading test is made. The assumption for scan conversion is that you are trying to render quickly, and hence most of the complex shading options are turned off. Note that in some images the quality will be improved if you turn off Normal correction.

If any of the three flags Shadow_Check, Reflect_Check, or Transmit_Check are explicitly set and scan conversion is used to render the scene, then at every pixel that is displayed, a recursive call to the raytracer will be made to determine the correct shading. Note that due to the faceted nature of objects

during scan conversion, shadowing, and especially refraction can get messed up during scan conversion. Should this happen, make the value of shadow_tolerance larger in the file polyray.ini.

For example if you want to do a scan conversion that contains shadows, you would use the following:

```
polyray xxx.pi -r 1 -q 1
```

or if you wanted to do raytracing with no shadows, reflection turned on, transparency turned off, and with diffuse and specular shading performed for both sides of surfaces you would use options 2 and 8:

```
polyray xxx.pi -q 10
```

Note: texturing cannot be turned off.

1.7.1 Raytracing

Raytracing is a very compute intensive process, but is the method of choice for a final image. The quality of the images that can be produced is entirely a function of how long you want to wait for results. There are certain options that allow you to adjust how you want to make tradeoffs between: quality, speed, and memory requirements.

The basic operation in raytracing is that of shooting a ray from the eye, through a pixel, and finally hitting an object. For each type of primitive there is specialized code that helps determine if a ray has hit that primitive. The standard way that Polyray generates an image is to use one ray per pixel and to test that ray against all of the objects in the data file.
Antialiasing or focal blur will result in more than one ray per pixel, increasing rendering time.

1.7.2 Scan Conversion

In order to support a quicker render of images, Polyray can render primitives as a series of polygons. Each polygon is scan converted using a Z-Buffer for depth information, and an image buffer for color information.

There are two variants of this type of rendering: full texturing at every pixel, and Gouraud shading. Gouraud shading performs a texturing calculation at the corners of all triangles and fills in color by interpolating between the vertices. The former looks much better for textured scenes, and slightly better if simple

opaque textures are used. Gouraud shading is the faster of the two techniques.

The scan conversion process does not by default provide support for: shadows, reflectivity, or transparency. It is possible to instruct Polyray to use raytracing to perform these shading operations through the use of either the global shade flags or by setting shade flags for a specific object. An alternative method for quickly testing shadows involves using depth mapped lights (see section 2.4.1.5). Reflections can be simulated with a multipass approach where an image map or environment map is calculated and then applied to the reflective object.

The memory requirements for performing scan conversion can be quite substantial. You need at least as much memory as is required for raytracing, plus at least 8 bytes per pixel for the final image. In order to correctly keep track of depth and color - 4 bytes are used for the depth of each pixel in the Z-Buffer (32 bit floating point number), and 4 bytes are used for each pixel in the S-Buffer (1 byte each for red, green, blue, and opacity).

During scan conversion, the number of polygons used to cover the surface of a primitive is controlled using the keywords u_steps, and v_steps (or combined with uv_steps). These two values control how many steps around and along the surface of the object values are generated to create polygons (see section 2.3.1.1.6). The higher the number of steps, the smoother the final appearance. Note however that if the object is very small then there is little reason to use a fine mesh - hand tuning is sometimes required to get the best balance between speed and appearance.

Blobs, polynomial functions and implicit functions are rendered by polygonalization of the surfaces by extracting an approximation to their isosurface. Currently the value of u_steps determines the number of slices along the x-axis, the value of v_steps controls the number of slices along the y-axis, and the value of w_steps controls the number of slices along the z-axis.

Polyray takes advantage of the virtual memory capabilities of Windows, allowing for larger images to be rendered as opposed to the older versions.

Also note that there are images that render slower in scan conversion and wireframe that raytracing! These are typically those files that contain large numbers of spheres and cones such as data files generated by CTDS or large gridded objects. The scan conversion process generates every possible part of each object, whereas the raytracer is able to sort the objects an only display the visible parts of the surfaces. If you run into this situation, a speedup trick you can use is to set the values of uv_steps very low for the small objects (e.g., uv_steps 6, 4 for spheres).

1.7.3 Wireframe

In most cases the fastest way to display a model is by showing a wireframe representation. Polyray supports this as a variant of the scan conversion process. When drawing a wireframe image only the edges of the polygons that are generated as part of scan conversion are drawn onto the screen.

There are two ways that a model can be rendered in wireframe. The first draws all edges and the second hides edges that are behind closer objects. In either case the edges are trimmed by any CSG operations.

1.7.4 Depth Files

Instead of creating an image, it is possible to instruct Polyray to create a file that contains depth information. Each pixel in the resulting Targa will contain the distance from the eye to the first surface that is hit. The format of the files is identical to the formats used by height fields (see 2.3.2.9.1).

For example, to create a 24 bit depth file, the following command could be used:

```
polyray sphere.pi -p 24 -d
```

Alternatively you can also use ‘image_format’ plus ‘pixelsize’ statements within the data file to tell Polyray to create a depth file with a specific bits per pixel. Both of these are modifiers to the viewpoint declaration (see section 2.2).

Polyray will disable any antialiasing when creating depth files (it is inappropriate to average depths).

There are three common applications for depth files: shadow maps (see Depth Lights, 2.4.1.5), height fields, and creating Random Dot Stereograms (RDS).

RDS images are an interesting way to create a 3D picture that doesn't require special glasses. The program RDSGEN (available in the tools directory on the Polyray github) will accept Polyray's 24 bit depth format and create an RDS image.

1.7.5 Raw Triangles

A somewhat odd addition to the image output formats for Polyray is the generation of raw triangle information. What happens is very similar to the scan conversion process, but rather than draw polygons, Polyray will write a text description of the polygons (after splitting them into triangles). The final output is a (usually long) list of lines, each line describing a single smooth triangle. The format of the output is one of the following:

```
   x1 y1 z1 x2 y2 z2 x3 y3 z3
or
   x1 y1 z1 x2 y2 z2 x3 y3 z3 nx1 ny1 nz1 nx2 ny2 nz2 nx3 ny3 nz3
   u1 v1 u2 v2 u3 v3
```

If the output is raw triangles then only the three vertices are printed. If uv_triangles are being created, then the normal information for each of the vertices follows the vertices and the u/v values follow them. The actual u/v values are specific to the type of object being generated.

Currently I don't have any applications for this output, but the first form is identical to the input format of the program RAW2POV. The intent of this feature is to provide a way to build models in polygon form for conversion to the input format of another renderer.

For example, to produce raw triangle output describing a sphere, and dump it to a file you could use the command:

```
   polyray sphere.pi -r 5 > sphere.tri
```

For full vertex, normal, and uv information, use the following command:

```
   polyray sphere.pi -r 6 > uvsphere.tri
```

For triangles that are trimmed by CSG operations, use the following command:

```
polyray sphere.pi -r 7 > uvsphere.tri
```

Nothing is drawn on screen while the raw triangles are generated, so typically you would turn off the graphics display (-V 0) when using this option.

CSG triangles are like raw triangles, but the triangles are further trimmed based on CSG (this can cause the creation of many more small triangles).

To tune the output of CSG triangles, you will want to set the values of the two variables, csg_tolerance, and csg_subdivisions, in POLYRAY.INI.
csg_tolerance is how finely a triangle will be diced in world coordinates to check for CSG, csg_subdivisions is how many recursive steps are taken with each triangle while checking for CSG. You want to keep csg_subdivisions low (like less than 5) as the number of CSG checks goes up like 4n where n is the number of subdivision levels. The defaults are:

```
csg_tolerance 0.05
csg_subdivision_depth 3
```

You would change csg_tolerance based on the world coordinate sizes of your model. If you have a model that extends from −1000 to 1000, you would probably set it around 10. The default assumes models that around 1 unit in size.

1.8 Display Options (IBM format only)

This section is deprecated but still included for legacy reasons and is not supported by the latest version. Future versions will support SDL. Polyray supports several VESA display modes. The table below lists the modes, the command line value to use with '-V x', and the entry in the file polyray.ini to set a default video mode.

Resolution	Colors	Command Line	Initialization name
N/A	none	-V 0	none
320x200	256	-V 1	vga/vga1
640x480	256	-V 2	vga2
800x600	256	-V 3	vga3
1024x768	256	-V 4	vga4
1280x1024	256	-V 5	vga5
320x200	32K	-V 6	hicolor/hicolor1

```
  640x480        32K        -V 7        hicolor2
  800x600        32K        -V 8        hicolor3
 1024x768        32K        -V 9        hicolor4
1280x1024        32K        -V 10       hicolor5

  320x200        64K        -V 11       16bit1
  640x480        64K        -V 12       16bit2
  800x600        64K        -V 13       16bit3
 1024x768        64K        -V 14       16bit4
1280x1024        64K        -V 15       16bit5

  320x200        16M        -V 16       truecolor1
  640x480        16M        -V 17       truecolor2
  800x600        16M        -V 18       truecolor3
 1024x768        16M        -V 19       truecolor4
1280x1024        16M        -V 20       truecolor5

  320x200         16        -V 21       4bit1
  640x480         16        -V 22       4bit2
```

It is pretty unlikely you have a board that will do all of the resolutions listed above. For that reason, Polyray will check to see if the requested mode is possible on your board. If not, another mode will be selected. The first priority is to find a mode with the same # of bits per pixel (even if the screen resolution is lower). The second priority is to drop to the next lower # of bits per pixel, starting at the highest resolution possible.

Setting the default display mode in polyray.ini is done by inserting a line of the form:

```
display hicolor1
```

To override a display mode from the command line you would do something like:

```
polyray sphere.pi -V 12
```

Note that using the line status display will mess up some of the higher graphics modes. If this bothers you, use the flag -t 1 to limit statistics to only startup and totals (or -t 0 for no statistics).

2 Detailed description of the Polyray input format

An input file describes the basic components of an image:

- A viewpoint that characterizes where the eye is, where it is looking and what its orientation is.
- Objects, their shape, placement, and orientation.
- Light sources, their placement and color.

Beyond the fundamentals, there are many components that exist as a convenience such as definable expressions and textures. This section of the document describes in detail the syntax of all of the components of an input file.

2.1 Expressions

There are six basic types of expressions that are used in Polyray:

- float: Floating point expression (e.g., 0.5, 2 * sin(1.33)). These are used at any point a floating point value is needed, such as the radius of a sphere or the amount of contribution of a part of the lighting model.
- vector: Vector valued expression (e.g., <0, 1, 0>, red, 12 * <2, sin(x), 17> + P). Used for color expressions, describing positions, describing orientations, etc.
- arrays: Lists of expressions (e.g., [0, 1, 17, 42], [<0,1,0>, <2 * sin(theta), 42, −4>, 2 * <3, 7, 2>])
- cexper: Conditional expression (e.g., x < 42).
- string: Strings used for file names or systems calls
- images: A Targa, PNG, or JPEG image.

The following sections describe the syntax for each of these types of expressions, as well as how to define variables in terms of expressions. See also the description of color maps, image maps (from which you can retrieve color or vector values), indexed maps, and height maps.

2.1.1 Numeric expressions

In most places where a number can be used (i.e. scale values, angles, RGB components, etc.) a simple floating point expression (float) may be used.
These expressions can contain any of the following terms:

-0.1, 5e-3, ab, ...	A floating point number or defined value
'(' float ')'	Parenthesised expression
float ^ float	Exponentiation, same as pow(x, y)
float * float	Multiplication
float / float	Division
float + float	Addition

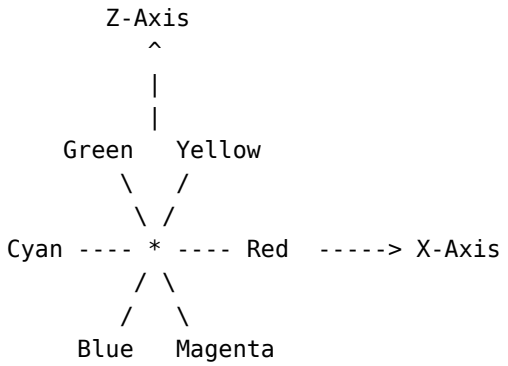
```
float - float           Subtraction
-float                  Unary minus
acos(float)             Arccosine, (radians for all trig functions)
asin(float)             Arcsin
atan(float)             Arctangent
atan2(float,float)      Angle from x-axis to the point (x, y)
bias(float,float)       Bias (see below)
ceil(float)             Ceiling function
cos(float)              Cosine
cosh(float)             Hyperbolic cosine
degrees(float)          Converts radians to degrees
exp(float)              e^x, standard exponential function
fabs(float)             Absolute value
floor(float)            Floor function
fmod(float,float)       Modulus function for floating point values
noise(vector)
noise(vector,vector)
gain(float,float)       Gain (see below)
heightmap(image,vector) Height of an pixel in an image
indexed_map(image,vector)   Index of a pixel in an image
cylindrical_indexed_map(image,vector)
spherical_indexed_map(image,vector)
legendre(l,m,n)         Legendre function
ln(float)               Natural logarithm
log(float)              Logarithm base 10
min(float,float)        Minimum of the two arguments
max(float,float)        Maximum of the two arguments
noise(vector)
noise(vector,float)     Solid texturing (noise) function.  If the
                        second argument is given, it is used as the
                        number of octaves (repetitions) of the 3D
                        noise function.
noise(vector,vector)    Second arg provides more flexible operation
                        using: <pos scale, noise scale, octaves>
pow(float,float)        Exponentiation (x^y)
radians(float)          Converts degrees to radians
ramp                    Ramp function (see below)
random                  Number between 0 and 1
sawtooth(float)         Sawtooth function (range is 0 - 1)
sin(float)              Sine
sinh(float)             Hyperbolic sine
sqrt(float)             Square root
tan(float)              Tangent
tanh(float)             Hyperbolic tangent
visible(vector,vector)  Returns 1 if second point visible from
                        first.
vector[i]               Extract component i from a vector (0<=i<=3)
vector . vector         Dot product of two vectors
```

`|float|`	Absolute value (same as fabs)
`|vector|`	Length of a vector

2.1.2 Vector Expressions

In most places where a vector can be used (i.e. color values, rotation angles, locations, …), a vector expression is allowed. The expressions can contain any of the following terms:

`vector + vector`	Addition
`vector - vector`	Subtraction
`vector * vector`	Cross product
`vector ^ vector`	Exterior product <a, b, c> ^ <x, y, z> = <a*x, b*y, c*z>
`vector * float`	Scaling of a vector by a scalar
`float * vector`	Scaling of a vector by a scalar
`vector / float`	Inverse scaling of a vector by a scalar
`brownian(vector,vector)`	Makes a random displacement of the first point by an amount proportional to the components of the second point
`brownian(vector)`	Random displacement of up to 0.1
`color_wheel(x, y, z)`	RGB color wheel using x and z (y ignored), the color returned is based on <x, z> using the chart below:



Intermediate colors are generated by interpolation.

`dnoise(vector)`	
`dnoise(vector,float)`	Returns a vector (gradient) based on the location given in the first argument. If the second argument is given, it is used as the number of octaves (repetitions) of the 3D noise function
`dnoise(vector,vector)`	Second arg provides more flexible operation using: <pos scale, noise scale, octaves>

`planar_imagemap(image, vector [, rflag])`
`cylindrical_imagemap(image, vector [, rflag])`

```
spherical_imagemap(image, vector [, rflag])
environment_map(vector, environment)
                        Image map lookup functions.  If the third
                        argument is given, then the image will be
                        tiled, otherwise black is used outside the
                        bounds of the image.  Note: for planar
                        image maps only the x and z coordinates of
                        the second argument are used.
ripple(vector,float,float)   (see below)
ripple(vector)
rotate(vector,vector)   Rotate the point specified in the first
                        argument by the angles specified in the
                        second argument (angles in degrees).
rotate(vector,vector,   Rotate the point specified in the first
       float)           argument about the axis specified in
                        the second argument by the angle given
                        in the third argument
reflect(vector,vector)  Reflect the first vector about the second
                        vector (particularly useful in environment
                        maps)
spline(type, t, [v0, v1, ..., vn], [p0, p1, ..., pn])
trace(vector,vector)    Color resulting from tracing a ray from the
                        the point given as the first argument in
                        the direction given by the second argument.
```

2.1.3 Arrays

Arrays are a way to represent data in a convenient list form. A good use for arrays is to hold a number of locations for polygon vertices or as locations for objects in successive frames of an animation.

As an example, a way to define a tetrahedron (4 sided solid) is to define its vertices, and which vertices make up its faces. By using this information in an object declaration, we can make a tetrahedron out of polygons very easily.

```
define tetrahedron_faces
   [<0, 1, 2>, <0, 2, 3>, <0, 3, 1>, <1, 3, 2>]

define tetrahedron_vertices
   [<0, 0, sqrt(3)>,
    <0, (2*sqrt(2)*sqrt(3))/3, -sqrt(3)/3>,
    <-sqrt(2), -(sqrt(2)*sqrt(3))/3, -sqrt(3)/3>,
    <sqrt(2), -(sqrt(2)*sqrt(3))/3, -sqrt(3)/3>]

define tcf tetrahedron_faces
define tcv tetrahedron_vertices
define tetrahedron
```

```
object {
 object { polygon 3,tcv[tcf[0][0]],tcv[tcf[0][1]],tcv[tcf[0][2]]} +
 object { polygon 3,tcv[tcf[1][0]],tcv[tcf[1][1]],tcv[tcf[1][2]]} +
 object { polygon 3,tcv[tcf[2][0]],tcv[tcf[2][1]],tcv[tcf[2][2]]} +
 object { polygon 3,tcv[tcf[3][0]],tcv[tcf[3][1]],tcv[tcf[3][2]]}
 }
```

What happened in the object declaration is that each polygon grabbed a series of vertex indices from the array tetrahedron_faces, then used that index to grab the actual location in space of that vertex.

Another example is to use an array to store a series of view directions so that we can use animation to generate a series of very distinct renders of the same scene (the following example is how the views for an environment map are generated):

```
define location <0, 0, 0>
define at_vecs [< 1, 0, 0>, <-1, 0, 0>, < 0, 1, 0>, < 0,-1, 0>,
                < 0, 0, 1>, < 0, 0,-1>]
define up_vecs [< 0, 1, 0>, < 0, 1, 0>, < 0, 0,-1>, < 0, 0, 1>,
                < 0, 1, 0>, < 0, 1, 0>]

// Generate six frames
start_frame 0
end_frame 5

// Each frame generates the view in a specific direction.  The
// vectors stored in the arrays at_vecs, and up_vecs turn the
// camera in such a way as to generate image maps correct for using
// in an environment map.
viewpoint {
   from location
   at location + at_vecs[frame]
   up up_vecs[frame]
   ...
   }
```

Sample files: tetrahed.pi, environ.pi

2.1.4 Descriptions of functions

Below are more detailed descriptions of some of the predefined Polyray functions.

2.1.4.1 Vector spline function

The spline function has the form:

```
spline(type, t, [v0, v1, …, vn], [p0, p1, …, pn])
```

It returns the value v0 if t <= 0, vn if t >= 1, and a value from a spline through the array of points for any value between 0 and 1. The first argument is optional and determines the type of the spline (defaults to type 0). The fourth parameter is also optional and is used to define what value of t corresponds to each vertex. If the parameter values are omitted then a chord length approximation is used (distance/time between vertices is about the same as the straight line distance between them).

Valid values for type and their meaning are:

0. • A cubic spline starting at the first control point and finishing at the last control point.
1. • A closed cubic spline (periodic, circular, …).
2. • A polyline between each of the control points
3. • A closed polyline (last control point is automatically connected to the first).
4. • A cubic spline with the derivatives defined for the endpoints. The syntax is:

```
spline(4, t, [d0, v0, v1, …, vn, dn], [p0, p1, …, pn])
```

Note that if the parameters are included, then there must be exactly two less entries in the parameter array than in the control point array.

For example, a flythrough can be performed with just a few control points and the spline function:

```
define cam_path [
 <0.384631,-0.229508,3>,
 <0.874141,-0.262296,0.03615>,
 <0.3147,-0.508196,-3.57831>,
 <3.321691,-0.131148,-0.25302>,
 <2.76225,-0.163934,1.12047>]

define cam_from spline(time,cam_path)
define cam_at spline(time+0.001,cam_path)

viewpoint {
   from cam_from
   at cam_at
   }
```

Note: The straight line spline is unlikely to be useful for a flythrough.
There will be sudden changes of direction of the camera at each control point.

2.1.4.2 fnoise function

A second noise function has been added, “fnoise()”. It is a variation on the existing noise() function that more closely matches the Perlin turbulence function. All the arguments are the same as noise(), i.e.:

```
fnoise(P)
fnoise(P, octaves)
fnoise(P, <pos scale, noise scale, octaves>)
```

2.1.4.3 Bias and gain functions

For tweaking the inputs to color maps, two functions have been added:

```
bias(a, t)
gain(a, t)
```

These two functions take input parameters a and t (both between 0 and 1) and generate an output between 0 and 1. The variable t modifies the shape of the curve. These functions are modelled after the ones described by Perlin.

The function bias pushes input values between 0 and 1 towards 0 if t is close to 0 and towards 1 if t is close to 1. If t is 0.5, then the output of bias is the same as the input. Use this if your color map values are to thin or to thick at one end or the other.

The function gain is an s-curve that starts flat and ends flat if t is close to 0 and starts vertical and ends vertical if t is close to 1. Like bias, if t is 0.5 it returns the input value without change.

Values of lower than 0 or greater than 1 will give undefined results.

These functions are particularly useful for manipulating color maps. For example, the following show how you can adjust the amount of clouds to blue in a sky map:

```
define cloudy_sky_map
      color_map(
         [0.0, 0.6, <0.4, 0.4, 0.4>, <1, 1, 1>]
         [0.6, 0.8, <1, 1, 1>, <0.196078, 0.6, 0.8>]
         [0.8, 1.0, <0.196078, 0.6, 0.8>, <0.196078, 0.6, 0.8>])
```

```
// Adjust the bias of the cloud function so that the sky gets
// cloudier as we look farther away.  Directly up is clear
define cloud_rate 0.7
define cloud_bias min(0.5 + bias(|I[1]|, cloud_rate), 1)

// Build the sky sphere
object {
   sphere <0, 0, 0>, 20000
   scale <1, 0.01, 1>
   texture {
      special surface {
         color cloudy_sky_map[bias(fnoise(P, 5), cloud_bias)]
         ambient 0.9
         diffuse 0
         specular 0
         }
      scale <500, 100, 300>
      }
   shading_flags 0
   }
```

2.1.4.4 Ripple function

The ripple function is used to modify normals as if they were being affected by ripples moving outward from a point.

```
      ripple(P),
      ripple(P, freq, phase),
```

To simulate the ripple function used in the standard textures, you can sum several ripple functions. For example, the declarations below are a close match for the blue_ripple texture:

```
define ripple_freq 20
define ripple_phase 0

define ripple_center1 2*(<random, random, random> - white/2)
define ripple_center2 2*(<random, random, random> - white/2)
define ripple_center3 2*(<random, random, random> - white/2)
define ripple_center4 2*(<random, random, random> - white/2)
define ripple_center5 2*(<random, random, random> - white/2)

// Piece together the centers to make an overall ripple
define ripple_fn
   (ripple(P, ripple_center1, ripple_freq, ripple_phase) +
    ripple(P, ripple_center2, ripple_freq, ripple_phase) +
    ripple(P, ripple_center3, ripple_freq, ripple_phase) +
    ripple(P, ripple_center4, ripple_freq, ripple_phase) +
    ripple(P, ripple_center5, ripple_freq, ripple_phase)) / 5
```

```
define blue_ripple
texture {
   special shiny {
      color <0.4, 0.4, 1.0>
      normal N + ripple_fn
      reflection 0.5
      }
   }
```

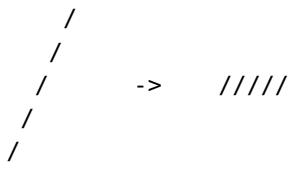
Another nice effect is to diminish the size of the ripples over distance. The example below shows two ripples that are strong at their centers and fade in effect farther away:

```
define C1 <0, 0, 0>
define C2 <2, 0, 2>
define ripple_fn
   0.2 * ripple(P, C1, 20, 0) * exp(-|C1-P|) +
   0.2 * ripple(P, C2, 10, 0) * exp(-|C2-P|)
```

2.1.4.5 Ramp function

ramp(x)

This chops a value so that it is always between 0 and 1. If a number is larger than 1 then the fractional part is returned. If the number is less than 0, the difference betwen it and the next lower number is returned. This gives the shape of a series of ramps:



This particular function is useful if you have a color map that covers the range from 0 to 1 and you want to ensure that the function you feed into it stays inside the map. For example, an onion texture could be defined with:

```
   define onion_fn ramp(|P|)
   define white_onion_map
       color_map([0, 0.8, white, 0.5*white]
                 [0.8, 1, 0.5*white, 0.2*white])
   define white_onion
      texture { special shiny { color white_onion_map[onion_fn] } }
```

If you are used to using the POV-Ray keywords frequency and phase in your textures, it’s a simple thing to get the same effect. Simply add them inside the ramp() function:

```
define onion_freq 42
define onion_phase 0
define onion_fn ramp(onion_freq * |P| + onion_phase)
```

2.1.5 Conditional Expressions

Conditional expressions are used in one of two places: conditional processing of declarations (see section 2.7) or conditional value functions.

cexper has one of the following forms:

```
!cexper
cexper && cexper
cexper || cexper
float < float
float <= float
float > float
float >= float
```

A use of conditional expressions is to define a texture based on other expressions, the format of this expression is:

```
(cexper ? true_value : false_value)
```

Where true_value/false_value can be either floating point or vector values.
This type of expression is taken directly from the equivalent in the C language. Unlike the C language, the parentheses around the conditional expression are required by Polyray. An example of how this is used (from the file spot1.pi) is:

```
special surface {
   color white
   ambient (visible(W, throw_offset) == 0
            ? 0
            : (P[0] < 1 ? 1
              : (P[0] > throw_length ? 0
                 : (throw_length - P[0]) / throw_length)))
   transmission (visible(W, throw_offset) == 1
                 ? (P[0] < 1 ? 0
                    : (P[0] > throw_length ? 1
                       : P[0] / throw_length))
                 : 1), 1
   }
```

In this case conditional statements are used to determine the surface characteristics of a cone defining the boundary of a spotlight. The amount of ambient light is modified with distance

from the apex of the cone, the visibility of the cone is modified based on both distance and on a determination if the cone is behind an object with respect to the source of the light.

2.1.6 Run-Time expressions

There are a few expressions that only have meaning during the rendering process:

```
I       Direction of the ray that struck the object
P       Point of intersection in object coordinates
N       Normal to the point of intersection in world
        coordinates
W       Point of intersection in world coordinates
U       The u/v coordinate of the intersection point
x,y,z   Components of the point in object coordinates
u,v   Components of the uv-coordinate of the point
w       The depth of recursion (reflections, refractions, ...)
```

These expressions describe the interaction of a ray and an object. To use them effectively, you need to understand the distinction between world coordinates, object coordinates, and u/v coordinates. Object coordinates describe a point or a direction with respect to an object as it was originally defined. World coordinates describe the same point after it has been rotated/scaled/translated. u/v coordinates describe the point in a way natural to a particular object type (e.g., latitude and longitude for a sphere). Typically texturing is done in either object coordinates or u/v coordinates so that as the object is moved around the texture will move with it. On the other hand shading is done in world coordinates.

The variables u and v are specific to each surface type and in general are the natural mapping for the surface (e.g., latitude and longitude on a sphere) In general u varies from 0 to 1 as you go around an object and v varies from 0 to one as you go from the bottom to the top of an object. These variables can be used in a couple of ways, to tell Polyray to only render portions of a surface within certain uv bounds, or they can be used as arguments to expressions in textures or displacement functions.

Not all primitives set meaningful values for u and v, those that do are:

```
bezier, cone, cylinder, disc, height fields, NURB, parabola,
parametric, sphere, torus, patch
```

Other surface types will simply have u=x, v=y. An intersection with a gridded object will use the u/v coordinates of the object within the grid.

See the file uvtst.pi in the data archives for an example of using uv bounds on objects. The file spikes.pi demonstrates using uv as variables in a displacement surface. The file bezier1.pi demonstrates using uv as variables to stretch an image over the surface of a bezier patch.

The meanings of some of these variables are slightly different when creating particle systems (see 2.3.5), or when coloring the background of an image (see 2.4.2)

2.1.7 Named Expressions

A major convenience for creating input files is the ability to create named definitions of surface models, object definitions, vectors, etc. The way a value is defined takes one of the following forms:

```
   define token expression          float, vector, array, cexper
   define token "str..."            string expression
   define token object { ... }
   define token surface { ... }
   define token texture { ... }
   define token transform { ... }   Each entry may be one of
                                    scale/translate/rotate/shear
   define token particle { ... }
```

Objects, surfaces, and textures can either be instantiated as-is by using the token name alone, or it can be instantiated and modified:

```
   token,
or
   token { ... modifiers ... },
```

Polyray keeps track of what type of entity the token represents and will parse the expressions accordingly.

Note: It is not possible to have two types of entities referred to by the same name. If a name is reused, then a warning will be printed, and all references to that name will use the new definition from that point on.

2.1.7.1 Static Variables

Static variables are a way to retain variable values (expressions, objects, textures, …) from frame to frame of an animation. Instead of the normal declaration of a variable:

```
define xyz 32 * frame
```

you would do something like this:

```
if (frame == start_frame)
   static define xyz 42
else
   static define xyz (xyz + 0.1)
```

The big differences between a static define and a define are that the static variable will be retained from frame to frame, and the static variable actually replaces any previous definitions rather than simply overloading them.

The static variables have an additional use beyond simple arithmetic on variables. By defining something that takes a lot of processing at parse time (like height fields and large image maps), you can make them static in the first frame and simply instantiate them every frame after that.

One example of this would be to spin a complex height field, if you have to create it every frame, there is a long wait while Polyray generates the field.
The following declarations would be a better way:

```
if (frame == start_frame)
   static define sinsf
      object {
         smooth_height_fn 128, 128, -2, 2, -2, 2,
                          0.25 * sin(18.85 * x * z + theta_offset)
         shiny_red
         }
...
sinsf { rotate <0, 6*frame, 0> }
...
```

Several examples of how static variables can be used are found in the animation directory in the data file archive. Two that make extensive use of static variables are movsph.pi, which bounces several spherical blob components around inside a box, and cannon.pi which points a cannon in several directions, firing balls once it is pointed.

Warning: A texture inside a static object should ONLY be static itself. The reason is that between frames, every non-static thing is deallocated. If you have things inside a static object that point to a deallocated pointer, you will most certainly crash the program. Sorry, but detecting these things would be too hard and reallocating all the memory would take up too much space.

2.1.7.2 Lazy Evaluation

Normally, Polyray tries to reduce the amount of time and space necessary for evaluating and storing variables and expressions. For example, if you had the following definition of x:

```
define x 2 * 3
```

Polyray would store 6 rather than the entire expression. This can cause problems in certain circumstances if Polyray decides to reduce the expression too soon. The problem is most notable in particle systems since the expression used in the particle declaration is used for the life of the particle system. An example of the problem would be a declaration like:

```
define t (frame - start_frame) / (end_frame - start_frame)
```

When Polyray encounters this it will store a value for t that is between 0 (for the first frame) and 1 (for the last frame). This great, up until you declare a particle system with something like:

```
if (frame == start_frame)
   define partx
   particle {
      ...
      death (t > 0.5 ? 1 : 0)
      }
```

Clearly the intent here is that the particles created by this system will die halfway through the animation (t > 0.5). This won't happen, since Polyray will evaluate the value of t at the time that partx is declared, and store the expression (0 > 0.5 ? 1 : 0), which then reduces to 0. This means the particle will never die.

To avoid this difficulty, the keyword noeval can be added to a definition to force Polyray to postpone evaluation. The declaration of t would now be:

```
define noeval t (frame - start_frame) / (end_frame - start_frame)
```

When partx is declared, the entire expression is now used and the particles will die at the appropriate time. Note that noeval is always the last keyword in a definition, and it works correctly with static definitions as well as normal ones.

Sample file: part6.pi

2.2 Definition of the viewpoint

The viewpoint and its associated components define the position and orientation the view will be generated from.

The format of the declaration is:

```
viewpoint {
   from vector
   at vector
   up vector
   angle float
   hither float
   resolution float, float
   aspect float
   yon float
   max_trace_depth float
   aperture float
   max_samples float
   focal_distance float
   image_format float
   pixel_encoding float
   pixelsize float
   antialias float
   antialias_threshold float
   }
```

All of the entries in the viewpoint declaration are optional and have reasonable default values (see below). The order of the entries defining the viewpoint is not important, unless you redefine some field. (In which case the last is used.)

The parameters are:

aspect	The ratio of width to height. (Default: 1.0.)
at	The center of the image, in world coordinates. (Default: <0, 0, 0>)
angle	The field of view (in degrees), from the center of the top row to the center of the bottom row. (Default: 45)
from	The location of the eye. (Default: <0, 0, -1>)
hither	Distance to front of view pyramid. Any intersection closer than this value will be

```
                      ignored. (Default: 1.0e-3)
resolution            Number of columns and rows in the image.
                      (Default: 256x256)
up                    Which direction is up (Default: <0, 1, 0>)
yon                   Distance to back of view pyramid.  Any
                      intersection beyond this distance will be
                      ignored. (Default: 1.0e5)
max_trace_depth       This allows you to tailor the amount of
                      recursion allowed for scenes with reflection
                      and/or transparency. (Default: 5)
aperture              If larger than 0, then extra rays are shot
                      (controlled by max_samples) to produce a
                      blurred image. Good values are between 0.1
                      and 0.5. (Default: 0)
max_samples           Number of rays/pixel when performing focal
                      blur (Default: 4)
focal_distance        Distance from the eye to the point that things
                      are in focus, this defaults to the distance
                      between from and at.
image_format          If 0 then normal image, if 1 then depth image
pixel_encoding        If 0 then uncompressed, if 1 then image is RLE
pixelsize             Number of bits/pixel in image (Default: 16)
antialias             Level of antialiasing to use (Default: 0)
antialias_threshold   Threshold to start antialiasing
                      (Default: 0.01)
```

The view vectors will be coerced so that they are perpendicular to the vector from the eye (from) to the point of interest (at).

A typical declaration is:

```
viewpoint {
   from <0, 5, -5>
   at   <0, 0,  0>
   up   <0, 1,  0>
   angle 30
   resolution 320, 160
   aspect 2
   }
```

In this declaration the eye is five units behind the origin, five units above the x-z plane and is looking at the origin. The up direction is aligned with the y-axis, the field of view is 30 degrees and the output file will default to 320x160 pixels.

In this example it is assumed that pixels are square, and hence the aspect ratio is set to width/height. If you were generating an image for a screen that has pixels half as wide as they are high then the aspect ratio would be set to one.

Note that you can change from left handed coordinates (the default for Polyray) to right handed by using a negative aspect ratio (e.g., aspect -4/3).

2.3 Objects/Surfaces

In order to make pictures, the light has to hit something. Polyray supports several primitive objects. The following sections give the syntax for describing the primitives, as well as how more complex primitives can be built from simple ones.

An object declaration is how polyray associates a surface with its lighting characteristics, and its orientation. This declaration includes one of the primitive shapes (sphere, polygon, …), and optionally: a texture declaration (set to a matte white if none is defined), orientation declarations, or a bounding box declaration.

The format the declaration is:

```
object {
   shape_declaration
   [texture_declaration]
   [translate/rotate/scale declarations]
   [subdivision declarations]
   [displacement declaration]
   [shading flag declaration]
   [bounding box declaration]
   [u/v bounds declaration]
   }
```

The following sub-sections describe the format of the individual parts of an object declaration. (Note: The shape declaration MUST be first in the declaration, as any operations that follow have to have data to work on.)

2.3.1 Object Modifiers

Object modifiers are statements within an object declaration that are used to move and/or change the shape of the object.

The declarations are processed in the order they appear in the declaration, so if you want to resize an object before moving it, you need to put the scale statement before the translation statement. The exception to this are displacement and u/v bounds. These act on the underlying shape and are applied prior to any other object modifiers, regardless of where they appear in the object declaration.

2.3.1.1 Position and Orientation Modifiers

The position, orientation, and size of an object can be modified through one of four linear transformations: translation, rotation, scaling, and shear.

Other modifications that can be made to the shape are displacement and u/v bounding. A collection of transformations can be collected into a single declaration and reused with the transform declaration. A typical example of this would be:

```
define tx1
transform {
   scale <2, 1, 4>
   rotate <45, 37, 0>
   translate obj_center // Where obj_center was defined before this
   }

object {
   sphere ...
   texture { ... }
   tx1 // Use the transform
   }

object {
   cone ...
   texture { ... }
   tx1 { translate <1, 2, 3> } // Use the transform with modifications
   }
```

2.3.1.1.1 Translation

Translation moves the position of an object by the number of units specified in the associated vector. The format of the declaration is:

```
translate <xt, yt, zt>
```

2.3.1.1.2 Rotation

Rotation revolves the position of an object about the x, y, and z axes (in that order). The amount of rotation is specified in degrees for each of the axes. The direction of rotations follows a left-handed convention: if the thumb of your left hand points along the positive direction of an axis, then the direction your fingers curl is the positive direction of rotation. (A negative aspect ratio in the viewpoint flips everything around to a right handed system.)

The format of the declaration is:

```
rotate <xr, yr, zr>
```

For example the declaration:

```
rotate <30, 0, 20>
```

will rotate the object by 30 degrees about the x axis, followed by 20 degrees about the z axis.

Remember: Left Handed Rotations.

2.3.1.1.3 Scaling

Scaling alters the size of an object by a given amount with respect to each of the coordinate axes. The format of the declaration is:

```
scale <xs, ys, zs>
```

Note that using 0 for any of the components of a scale is a bad idea. It may result in a division by zero in Polyray, causing a program crash. Use a small number like 1.0e-5 to flatten things. Usually you will just want to use 1 for components that don’t need scaling.

2.3.1.1.4 Shear

A less frequently used, but occasionally useful transformation is linear shear. Shear scales along one axis by an amount that is proportional to the location on another axis. The format of the declaration is:

```
shear yx, zx, xy, zy, xz, yz
```

Typically only one or two of the components will be non-zero, for example the declaration:

```
shear 0, 0, 1, 0, 0, 0
```

will shear an object more and more to the right as y gets larger and larger.
The order of the letters in the declaration is descriptive, shear … ab, … means shear along direction a by the amount ab times the position b.

This declaration should probably be split into three: one that associates shear in x with the y and z values, one that associates shear in y with x and z values, and one that associates shear in z with x and y values.

You might want to look at the file xander.pi - this uses shear on boxes to make diagonally slanted parts of letters.

2.3.1.1.5 Displace

The displacement operation causes a modification of the shape of an object as it is being rendered. The amount and direction of the displacement are specified by the statement:

```
   displace vector
or
   displace float
```

If a vector expression is given, then Polyray will displace each vertex in the surface by the vector. If a floating point expression is given, then Polyray will displace the surface along the direction of the normal, by the amount given in the expression.

For effective results, the value of u_steps and v_steps should be set fairly high. Polyray subdivides the surface, then performs the displacement. If there are too few subdivisions of the surface then the result will be a very coarse looking object.

An example of an object that has a displacement is shown below. The displacement in the outer object is applied to each of the two spheres within.

```
object {
   object {
      sphere <0,-1.5, 0>, 2
      u_steps 32
      v_steps 64
      shiny_red
      }
   + object {
      sphere <0, 1.5, 0>, 2
      u_steps 32
      v_steps 64
      shiny_blue
      translate <0, -1.5, 0>
      rotate <-20, 0, 30>
      translate <0, 2.25, 0>
      }
   displace 0.5 * sin(5*y)
   }
```

Sample Files: disp2.pi, disp3.pi, legen.pi, spikes.pi

2.3.1.1.6 UV Bounds

By adjusting the value of the u/v bounds, it is possible to only render a selected portion of a surface. The format of the declaration is:

```
uv_bounds low_u, high_u, low_v, high_v
```

For example, the following declaration will result in a wedge shaped portion of a sphere:

```
object {
   sphere <-2, 2, 0>, 1
   uv_bounds 0.3, 1.0, 0.0, 0.8
   }
```

The same effect could be achieved through the use of CSG with one or more clipping planes. For most purposes (e.g., creating a hemisphere by setting u_low to 0.5) the uv_bounds will easier to create and faster to render.

Sample File: uvtst.pi

2.3.1.2 Bounding box

In order to speed up the process of determining if a ray intersects an object and in order to define good bounds for surfaces such as polynomials and implicit surfaces, a bounding box can be specified. A short example of how it is used to define bounds of a polynomial:

```
define r0 3
define r1 1
define torus_expression (x^2 + y^2 + z^2 - (r0^2 + r1^2))^2 -
                        4 * r0^2 * (r1^2 - z^2)
object {
   polynomial torus_expression
   shiny_red
   bounding_box <-(r0+r1), -(r0+r1), -r1>,
                <  (r0+r1),  (r0+r1),  r1>
   }
```

The test for intersecting a ray against a box is much faster than performing the test for the polynomial equation. In addition the box helps the scan conversion process determine where to look for the surface of the torus.

2.3.1.3 Subdivision of Primitives

The amount of subdivision of a primitive that is performed before it is displayed as polygons is tunable. These declarations are

used for scan conversion of object, when creating displacement surfaces, and to determine the quality of an implicit function. The declarations are:

```
u_steps n
v_steps m
w_steps l
uv_steps n, m
uv_steps n, m, l
```

Where u generally refers to the number of steps around the primitive (the number of steps around the equator of a sphere for example). The parameter v refers to the number of steps along the primitive (latitudes on a sphere). Cone and cylinder primitives only require 1 step along v, but for smoothness may require many steps in u.

For blobs, polynomials, and implicit surfaces, the u_steps component defines how many subdivisions along the x-axis, the v_steps component defines how many subdivisions along the y-axis, and the w_steps component defines how many subdivision along the z-axis.

2.3.1.4 Shading Flags

It is possible to tune the shading that will be performed for each object.
The values of each bit in the flag has the same meaning as that given for
global shading in section 1.7.1.4:

By default, all objects have the following flags set: Shadow_Check, Reflect_Chect, Transmit_Check, UV_Check, and Cast_Shadow. The two sides check is normally off.

The declaration has the form:

```
shading_flags xx
```

For example, if the value 82 (64 + 16 + 2) is used for xx above, then this object can be reflective and will cast shadows, however there will be no tests for transparency, there will be no shading of the back sides of surfaces, and there will be no shadows on the surface.

Note: the shading flag only affects the object in which the declaration is made. This means that if you want the shading values affected for all parts if a CSG object, then you will need a declaration in every component.

2.3.2 Primitives

Primitives are the lowest level of shape description. Typically a scene will contain many primitives, appearing either individually or as aggregates using Constructive Solid Geometry (CSG) operations or gridded objects.

Descriptions of each of the primitive shapes supported by Polyray, as well as references to data files that demonstrate them are given in the following subsections. Following the description of the primitives are descriptions of how CSG and grids can be built.

2.3.2.1 Bezier and bicubic patches

A Bezier patch is a form of bicubic patch that approximates its control vertices. The patch is defined in terms of a 4x4 array of control vertices, as well as several tuning values. Starting in v1.8 this primitive can also be used for a few other types of bicubic splines.

The format of the declaration is:

```
bezier patch_type, flatness_value,
       u_subdivisions, v_subdivision,
       [ 16 comma-separated vertices, i.e.
          <x0, y0, z0>, <x1, y1, z1>, ..., <x15, y15, z15> ]
```

The value of patch_type determines what basis is used for the patch. The kind of patch and the actual basis (for people that need to properly write the control vertices) are shown in the table below:

Type	Basis name	Basis matrix
0 -> 3	Bezier	{{-1, 3,-3, 1}, { 3,-6, 3, 0}, {-3, 3, 0, 0}, { 1, 0, 0, 0}}
4	B-Spline	{{-1, 3,-3, 1}, { 3,-6, 3, 0}, {-3, 0, 3, 0}, { 1, 4, 1, 0}}
5	Catmull-Rom	{{-1, 3,-3, 1}, { 2,-5, 4,-1}, {-1, 0, 1, 0}, { 0, 2, 0, 0}}
6	Hermite	{{ 2,-2, 1, 1}, {-3, 3,-2,-1}, { 0, 0, 1, 0}, { 1, 0, 0, 0}}

flatness_value is not currently used by Polyray. It is retained in the declaration for backwards compatibility as well as for possible future use by an adaptive subdivision algorithm.

The number of levels of subdivision of the patch, in each direction, is controlled by either the u_subdivisions and v_subdivisions given in the bezier shape declaration (or by the value of uv_steps - see section 2.3.1.3). The more subdivisions allowed, the smoother the approximation to the patch, however storage and processing time go up.

An example of a bezier patch is:

```
object {
   bezier 2, 0.05, 3, 3,
      <0, 0, 2>, <1, 0, 0>, <2, 0, 0>, <3, 0,-2>,
      <0, 1, 0>, <1, 1, 0>, <2, 1, 0>, <3, 1, 0>,
      <0, 2, 0>, <1, 2, 0>, <2, 2, 0>, <3, 2, 0>,
      <0, 3, 2>, <1, 3, 0>, <2, 3, 0>, <3, 3,-2>
uv_steps 8, 8
rotate <30, -70, 0>
shiny_red
}
```

Sample files: bezier0.pi, teapot.pi, teapot.inc

2.3.2.2 Blob

A blob describes a smooth potential field around one or more spherical, cylindrical, or planar components.

The format of the declaration is:

```
blob threshold:
   blob_component1
   [, blob_component2 ]
   [, etc. for each component ]
```

The threshold is the minimum potential value that will be considered when examining the interaction of the various components of the blob. Each blob component one of two forms:

```
sphere <x, y, z>, strength, radius
cylinder <x0, y0, z0>, <x1, y1, z1>, strength, radius
plane <nx, ny, nz>, d, strength, dist
torus <x, y, z>, <dx, dy, dz>, major, strength, minor
```

The strength component describes how strong the potential field is around the center of the component, the radius component describes the maximum distance at which the component will interact with

other components. For a spherical blob component the vector <x,y,z> gives the center of the potential field around the component. For a cylindrical blob component the vector <x0, y0, z0> defines one end of the axis of a cylinder, the vector <x1, y1, z1> defines the other end of the axis of a cylinder. A planar blob component is defined by the standard plane equation with <nx, ny, nz> defining the normal and 'd' defining the distance of the plane from the origin along the normal.

Note: The ends of a cylindrical blob component are given hemispherical caps.

Note: toroidal blob components won't render correctly in raytracing. The numerical precision of a PC is insufficient. It will work correctly in scan conversion or raw triangle output.

Note: The colon and the commas in the declaration really are important.

An example of a blob is:

```
object {
      blob 0.5:
         cylinder <0,  0, 0>, <5, 0, 0>, 1, 0.7,
         cylinder <1, -3, 0>, <3, 2, 0>, 1, 1.4,
         sphere <3, -0.8, 0>, 1, 1,
         sphere <4,  0.5, 0>, 1, 1,
         sphere <1,  1,   0>, 1, 1,
         sphere <1,  1.5, 0>, 1, 1,
         sphere <1,  2.7, 0>, 1, 1

      shiny_red
   }
```

Note: since all blob components (except toroidal) are a collection of 4th order polynomials, it is possible to specify which quartic root solver to use.
See section 2.3.2.15, for a description of the root_solver statement.

Sample files: blob.pi, tblob.pi

2.3.2.3 Box

A box is rectangular solid that has its edges aligned with the x, y, and z axes. it is defined in terms of two diagonally opposite

corners. The alignment can be changed by rotations after the shape declaration.

The format of the declaration is:

```
box <x0, y0, z0>, <x1, y1, z1>
```

Usually the convention is that the first point is the front-lower-left point and the second is the back-upper-right point. The following declaration is four boxes stacked on top of each other:

```
define pyramid
object {
     object { box <-1, 3, -1>, <1, 4, 1> }
   + object { box <-2, 2, -2>, <2, 3, 2> }
   + object { box <-3, 1, -3>, <3, 2, 3> }
   + object { box <-4, 0, -4>, <4, 1, 4> }
   matte_blue
   }
```

Sample file: boxes.pi

2.3.2.4 Cone

A cone is defined in terms of a base point, an apex point, and the radii at those two points. Note that cones are not closed (you must use discs to cap them).

The format of the declaration is:

```
cone <x0, y0, z0>, r0, <x1, y1, z1>, r1
```

An example declaration of a cone is:

```
object {
   cone <0, 0, 0>, 4, <4, 0, 0>, 0
   shiny_red
   }
```

Sample file: cone.pi

2.3.2.5 Cylinder

A cylinder is defined in terms of a bottom point, a top point, and its radius.
Note that cylinders are not closed.

The format of the declaration is:

```
cylinder <x0, y0, z0>, <x1, y1, z1>, r
```

An example of a cylinder is:

```
object {
   cylinder <-3, -2, 0>, <0, 1, 3>, 0.5
   shiny_red
   }
```

Sample file: cylinder.pi

2.3.2.6 Disc

A disc is defined in terms of a center a normal and either a radius, or using an inner radius and an outer radius. If only one radius is given, then the disc has the appearance of a (very) flat coin. If two radii are given, then the disc takes the shape of an annulus (washer) where the disc extends from the first radius to the second radius. Typical uses for discs are as caps for cones and cylinders, or as ground planes (using a really big radius).

The format of the declaration is:

```
   disc <cx, cy, cz>, <nx, ny, nz>, r
or
   disc <cx, cy, cz>, <nx, ny, nz>, ir, or
```

The center vector <cx,cy,cz> defines where the center of the disc is located, the normal vector <nx,ny,nz> defines the direction that is perpendicular to the disc. i.e. a disc having the center <0,0,0> and the normal <0,1,0> would put the disc in the x-z plane with the y-axis coming straight out of the center.

An example of a disc is:

```
object {
   disc <0, 2, 0>, <0, 1, 0>, 3
   rotate <-30, 20, 0>
   shiny_red
   }
```

Note: a disc is infinitely thin. If you look at it edge-on it will disappear.

Sample file: disc.pi

2.3.2.7 Glyphs (TrueType fonts)

The glyph primitive creates shapes similar to the sweep primitive. The big difference is that straight line and curved line segments may appear in the same contour. Additionally, the glyph can be made from several contours with exterior ones defining the outsides of the shape and interior ones defining holes within the glyph.

Typically the glyph primitive will be used in conjunction with the utility TTFX (described below) to translate TrueType font information into Polyray glyph information.

The following declaration creates a box with a square hole cut out of it.

```
object {
   glyph 2
      contour 4,
         <0, 0>, <4, 0>, <4, 4>, <0, 4>
      contour 4,
         <1, 1>, <1, 3>, <3, 3>, <3, 1>
   texture { shiny { color red reflection 0.2 } }
   translate <-2, 0, 0>
   }
```

The default placement of glyph coordinates is in the x-y plane and the glyph has a depth of one in z (starting at z=0 and going to z=1). To change the depth, just use something like: scale <1, 1, 0.2>.

Each entry in a contour is a 2D point. If a non-zero z is used, then the point is assumed to be an off-curve point and will create a curved segment within the contour. For example, the following declaration makes a somewhat star shaped contour with sharp corners for the inner parts of the star and rounded curves for the outer parts of the star:

```
object {
   glyph 1 contour 14,
     <r0*cos( 1*dt),r0*sin( 1*dt)>, <r1*cos( 2*dt),r1*sin( 2*dt),1>,
     <r0*cos( 3*dt),r0*sin( 3*dt)>, <r1*cos( 4*dt),r1*sin( 4*dt),1>,
     <r0*cos( 5*dt),r0*sin( 5*dt)>, <r1*cos( 6*dt),r1*sin( 6*dt),1>,
     <r0*cos( 7*dt),r0*sin( 7*dt)>, <r1*cos( 8*dt),r1*sin( 8*dt),1>,
     <r0*cos( 9*dt),r0*sin( 9*dt)>, <r1*cos(10*dt),r1*sin(10*dt),1>,
     <r0*cos(11*dt),r0*sin(11*dt)>, <r1*cos(12*dt),r1*sin(12*dt),1>,
     <r0*cos(13*dt),r0*sin(13*dt)>, <r1*cos(14*dt),r1*sin(14*dt),1>
   }
```

The program TTFX.EXE has been included (in the archive PLYUTL.ZIP) to help with the conversion of TrueType fonts from their .TTF format (typically found in the or directory) into the sort of declaration that Polyray can understand. For example,

```
   ttfx \windows\system\times.ttf Foo > temp.pi
or,
   ttfx \windows\system\times.ttf Foo 0.2 > temp.pi
```

By default, the characters in the string being converted (the word Foo above) are packed right next to each other. If a number follows the text string, then the characters are separated by that amount. A spacing of 0.2 has worked pretty well for the fonts I've tried.

If you then add the following line to the top of temp.pi (leaving everything else like viewpoint to their defaults)

```
light <0, 100, -100>
```

and render it with

```
polyray temp.pi -r 1
```

you will get a nice rendered image of the word Foo.

Note that the combination of TTFX, Polyray raw triangle output, and RAW2POV is a way you can create TrueType characters for use in a number of renderers.

Sample files: timestst.pi, glyph1.pi, glyph2.pi, glyph3.pi, wingtst.pi

2.3.2.8 Implicit Surface

The format of the declaration is:

```
function f(x,y,z)
```

The function f(x,y,z) may be any expression composed of the variables: x, y, z, a numerical value (e.g., 0.5), the operators: +, -, *, /, ^, and any Polyray supported function. The code is not particularly fast, nor is it totally accurate, however the capability to ray-trace such a wide class of functions by a SW program is (I believe) unique to Polyray.

The following object is taken from sombrero.pi and is a surface that looks very much like diminishing ripples on the surface of water.

```
define a_const 1.0
define b_const 2.0
define c_const 3.0
define two_pi_a 2.0 * 3.14159265358 * a_const

// Define a diminishing cosine surface (sombrero)
object {
   function y - c_const * cos(two_pi_a * sqrt(x^2 + z^2)) *
                          exp(-b_const * sqrt(x^2 + z^2))
   matte_red
```

```
   bounding_box <-4, -4, -4>, <4, 4, 4>
   }
```

Rendering the following object will show all the places within a sphere where the solid noise function takes on the value 0.5.

```
object {
   object {
      function noise(3*P) - 0.5
      u_steps 64
      v_steps 64
      w_steps 64
      Lapis_Lazuli
      bounding_box <-2, -2, -2>, <2, 2, 2>
      }
   & object { sphere <0, 0, 0>, 2 }
   rotate <-10, 20, 0>
   }
```

It is quite important to have the bounding box declaration within the object where the function is declared. If that isn't there, Polyray will be unable to determine where to look for intersections with the surface. (The bounding box of an implicit function defaults to <−1, −1, −1>, <1, 1, 1>.)

Sample files: noisefn.pi, sinsurf.pi, sectorl.pi, sombrero.pi, superq.pi, zonal.pi

2.3.2.9 Height Field

There are two ways that height fields can be specified, either by using data stored in a Targa file, or using an implicit function of the form y = f(x, z).

The default orientation of a height field is that the entire field lies in the square 0 <= x <= 1, 0 <= z <= 1. File based height fields are always in this orientation, implicit height fields can optionally be defined over a different area of the x-z plane. The height value is used for y.

2.3.2.9.1 File Based Height Fields

Height fields data can be read from any Targa, PNG, or JPEG format file.

Note that if you use JPEG images, a grayscale JPEG will be treated as an 8 bit format and a color JPEG will be treated as a 24 bit format height field. Due to the lossy nature of JPEG, it is extremely

unlikely that a color JPEG will be useful as a height field. It is possible that reasonable results can be obtained using grayscale JPEG images as height fields (no guarantees).

By using smooth_ in front of the declaration, an extra step is performed that calculates normals to the height field at every point within the field. The result of this is a greatly smoothed appearance, at the expense of around three times as much memory being used.

The format of the declaration is:

```
height_field "filename"
smooth_height_field "filename"
```

The sample program wake.c generates a Targa file that simulates the wake behind a boat.

Sample files: wake.pi, spiral.pi, islands.pi

For a description of how image files are interpreted as depths, see section 3.2.2 Depth map formats.

2.3.2.9.2 Implicit Height Fields

Another way to define height fields is by evaluating a mathematical function over a grid. Given a function y = f(x, z), Polyray will evaluate the function over a specified area and generate a height field based on the function. This method can be used to generate images of many sorts of functions that are not easily represented by collections of simpler primitives.

The valid formats of the declaration are:

```
height_fn xsize, zsize, minx, maxx, minz, maxz, expression
height_fn xsize, zsize, expression
smooth_height_fn xsize, zsize, minx, maxx, minz, maxz, expression
smooth_height_fn xsize, zsize, expression
```

If the four values minx, maxx, minz, and maxz are not defined then the default square 0 <= x <= 1, 0 <= z <= 1 will be used.

For example,

```
// Define constants for the sombrero function
define a_const 1.0
define b_const 2.0
define c_const 3.0
define two_pi_a 2.0 * 3.14159265358 * a_const
```

```
// Define a diminishing cosine surface (sombrero)
object {
   height_fn 80, 80, -4, 4, -4, 4,
      c_const * cos(two_pi_a * sqrt(x^2 + z^2)) *
                exp(-b_const * sqrt(x^2 + z^2))
   shiny_red
   }
```

will build a height field 80x80, covering the area from −4 <= x <= 4, and −4 <= z <= 4.

Compare the run-time performance and visual quality of the sombrero function as defined in sombfn.pi with the sombrero function as defined in sombrero.pi.
The former uses a height field representation and renders quite fast. The latter uses a very general function representation and gives smoother but very slow results.

Sample file: sombfn.pi, sinfn.pi

2.3.2.9.3.1 Image based Spherical Height Fields

The format of the declaration is:

```
sheight_field "filename"
sheight_field "filename", scale, base_radius
```

or, for automated smoothing:

```
smooth_sheight_field "filename"
smooth_sheight_field "filename", scale, base_radius
```

In order to create an spherical height map that is seamless, you need to distort the image so that it wraps properly onto a sphere. In the example below, a solid noise function is used to create a seamless height map. The definition pos is created by turning each pixel in the background into a point on a sphere. To create other height maps from solid texture functions, you would simply adjust the values of xres and yres, and plug the variable pos into your texturing function (which is a simple noise function in the example below).

```
define xres 721
define yres 361

define phi_width_adjust yres / (yres - 1)
define theta_width_adjust xres / (xres - 1)

viewpoint { resolution xres, yres }
```

```
define pi 3.14159
define phi pi * (phi_width_adjust * v - 0.5)
define theta 2 * pi * theta_width_adjust * u
define radius 2
define posx radius*cos(theta)*cos(phi)
define posy radius*sin(phi)
define posz radius*sin(theta)*cos(phi)
define pos <posx,posy,posz>

define land_fn fnoise(1.2*pos, 3) // 5)
define land_altitude_map
   color_map([0.0, 0.3, white, black]  // From mountains down to sea
             [0.3, 1.0, black, black]) // Ocean floor is flat...

background land_altitude_map[land_fn]
```

2.3.2.9.3.2 Implicit Spherical Height Fields

The format of the declaration is:

```
sheight_fn ures, vres, exper
sheight_fn ures, vres, exper, scale, base_radius
```

or, for automated smoothing:

```
smooth_sheight_fn ures, vres, exper
smooth_sheight_fn ures, vres, exper, scale, base_radius
```

Like the parametric surface, the spherical height field is a function of the variables u, and v. In the example below the variables u and v are translated into angles theta and phi. These variables are then used to create ripples waves over the surface of the height field.

```
define phi (v - 0.5) * 3.14159265
define theta u * 6.28318530

define ures 120
define vres 60

object { smooth_sheight_fn ures, vres,
            0.1 * cos(12 * phi) + 0.1 * cos(12 * theta), 1.0, 1.0 }
```

2.3.2.9.3 Spherical Height Fields

Spherical height fields work by moving points inward and outward from the surface of a sphere. Both image based and implicit versions are available.

In either case, there is a base radius that is used for the sphere. The heights are used as offsets from the base radius.

A scaling factor can be applied to the heights. The scaling is most useful when you are working with image based spherical height fields. An indexed image or a greyscale image will have depths that range from −128 to 127. To use these depths on a sphere that has a base radius of 1.0, you would use a scale the depths by 1/128. This results in a spherical height field that goes from a radius of 0 to a radius of just under 2.0.

For example,

```
object {
   sheight_field "bumpy.tga", 1/256, 1.5
   rotate <0, 60, 0>
   shiny_red
   }
```

results in a spherical height field that ranges from a radius of 1.0 to a radius of 2.0.

Sample files: shtst1.pi -> shtst10.pi

2.3.2.9.4.1 File Based Cylindrical Height Fields

The format of the declaration is:

```
cheight_field "filename"
cheight_field "filename", scale, base_radius
```

or, for automated smoothing:

```
smooth_cheight_field "filename"
smooth_cheight_field "filename", scale, base_radius
```

2.3.2.9.4.2 Implicit Cylindrical Height Fields

The format of the declaration is:

```
cheight_fn ures, vres, exper
cheight_fn ures, vres, exper, scale, base_radius
```

or, for automated smoothing:

```
smooth_cheight_fn ures, vres, exper
smooth_cheight_fn ures, vres, exper, scale, base_radius
```

Like spherical height fields, the cylindrical height fields are defined in terms of the variables u and v. For example, in the example below, the function is a set of ripples that are placed onto the surface of a cylinder (see chtst7.pi in the data archives).

The resolution of the height field is 240x240 and the normals are averaged to give a smoother appearance.

```
define ripple_fn
   |ripple(P1, pt1, ripple_freq, ripple_phase) * exp1 +
    ripple(P1, pt2, ripple_freq, ripple_phase) * exp2 +
    ripple(P1, pt3, ripple_freq, ripple_phase) * exp3 +
    ripple(P1, pt4, ripple_freq, ripple_phase) * exp4 +
```



```
    ripple(P1, pt5, ripple_freq, ripple_phase) * exp5 +
    ripple(P1, pt6, ripple_freq, ripple_phase) * exp6 +
    ripple(P1, pt7, ripple_freq, ripple_phase) * exp7 +
    ripple(P1, pt8, ripple_freq, ripple_phase) * exp8 |

object {
   smooth_cheight_fn 240, 240,
              white, white, ripple_fn,
              1.0, rlen
   translate <0,-0.5,0>
   scale <1,12,1>
   scale 0.5*white
   rotate <0, 120, 0>
   rotate <-20, 0, 0>
   }
```

2.3.2.9.4 Cylindrical Height Fields

Cylindrical height fields work by moving points inward and outward from the surface of a cylinder. Both image based and implicit versions are available.
The cylinder is oriented with one end at <0,0,0> and the other end at <0,1,0>.
The base width is 1.0 and the scale_factor is 1/127, unless otherwise specified (similarly to spherical height fields).

Sample files: chtst1.pi, chtst2.pi, …

2.3.2.10 Hypertexture

The format of the declaration is:

```
    hypertexture exper
```

This is an experimental hypertexture primitive. It doesn’t yet interact properly with other primitives. You may carve pieces out of it with CSG, however if other primitives extend inside one, they may disappear.

This primitive uses the uv_steps to step a ray through a volume of space. The overall size of the hypertexture is dependent on the bounding_box declaration.
At each step through the hypertexture, the color and opacity of the object texture are evaluated. The accumulation of color and opacity determine the overall shading for the pixel.

For example, the following defines a slightly whispy, colored sphere. Unlike a normal sphere, this object doesn’t have a surface, only a slightly opaque interior. Note the use of the bounding_box declaration, as well as the use of htexfn to limit the visible portions of the object.

```
define htexfn x^2 + y^2 + z^2 - 4

define htexcolor
   (htexfn > 0
    ? <0, 0, 0, 1>
    : color_map([0.0, 0.5, red, 0.2, violet, 0.4]
                [0.5, 1.0, violet, 0.4, green, 0.8])[noise(2*P)])

define spherical_red
texture {
   special surface {
      color htexcolor
      ambient 0
      diffuse 0.4
      specular white, 0.01
      microfacet Cook 10
      }
   }

object {
   hypertexture htexfn
   bounding_box <-2, -2, -2>, <2, 2, 2>
   spherical_red
   uv_steps 32, 32
   shading_flags 0
   rotate <0, frame * 10, 0>
   }
```

2.3.2.11 Lathe surfaces

A lathe surface is a polygon that has been revolved about the y-axis. This surface allows you to build objects that are symmetric about an axis, simply by defining 2D points.

The format of the declaration is:

```
lathe type, direction, total_vertices,
   <vert1.x,vert1.y,vert1.z>
   [, <vert2.x, vert2.y, vert2.z>]
   [, etc. for total_vertices vertices]
```

The value of type can be 1, 2, or 3. If the value is 1, then the surface will simply revolve the line segments. If the value is 2, then the surface will be represented by a spline that approximates the line segments that were given.
A lathe surface of type 2 is a very good way to smooth off corners in a set of line segments. If the value is 3, then the z component of each vertex is used to determine if it (the vertex) is on or off the curve (non-zero z for off-curve).

The value of the vector direction is used to change the orientation of the lathe. For a lathe surface that goes straight up and down the y-axis, use <0, 1, 0> for direction. For a lathe surface that lies on the x-axis, you would use <1, 0, 0> for the direction.

Sample file: lathe1.pi, lathe2.pi, lathe3.pi

Note that CSG will really only work correctly if you close the lathe - that is either make the end point of the lathe the same as the start point, or make the x-value of the start and end points equal zero. Lathes, unlike polygons are not automatically closed by Polyray.

Note: since a splined lathe surface (type = 2) is a 4th order polynomial, it is possible to specify which quartic root solver to use. See section 2.3.2.15 for a description of the root_solver statement.

2.3.2.12 NURBS

Polyray supports the general patch type, Non-Uniform Rational B-Splines (NURBS). All that is described here is how they are declared and used in Polyray. For further background and details on NURBS, refer to the literature.

They are declared with the following syntax:

```
nurb u_order, u_points, v_order, v_points,
     u_knots, v_knots, uv_mesh
```

or

```
nurb u_order, u_points, v_order, v_points, uv_mesh
```

Where each part of the declaration has the following format and definition,

Name	Format	Definition
u_order	integer	One more than the power of the spline in the u direction. (If u_order = 4, then it will be a cubic patch.)
u_points	integer	The number of vertices in each row of the patch mesh
v_order	integer	One more than the power of the spline in v direction.
v_points	integer	The number of rows in the patch mesh
u_knots	[…]	Knot values in the u direction
v_knots	[…]	Knot values in the v direction
uv_mesh	[…]	An array of arrays of vertices. Each vertex may have either three or four components. If the fourth component is set then the spline will be rational. If the vertex has only three components then the homogenous (fourth) component is assumed to be one. The homogenous component must be greater than 0.

For example, the following is a complete declaration of a NURB patch

```
object {
 nurb 4, 6, 4, 5,
      [0, 0, 0, 0, 1.5, 1.5, 3, 3, 3, 3], // Non-uniform knots
      [0, 0, 0, 0, 1, 2, 2, 2, 2],        // Uniform open knots
    [[<0,0,0>, <1,0, 3>, <2,0,-3>,     <3,0, 3>, <4,0,0>],
     [<0,1,0>, <1,1, 0>, <2,1, 0>,     <3,1, 0>, <4,1,0>],
     [<0,2,0>, <1,2, 0>, <2,2, 5,2>,   <3,2, 0>, <4,2,0>],
     [<0,3,0>, <1,3, 0>, <2,3, 5,0.5>, <3,3, 0>, <4,3,0>],
     [<0,4,0>, <1,4, 0>, <2,4, 0>,     <3,4, 0>, <4,4,0>],
     [<0,5,0>, <1,5,-3>, <2,5, 3>,     <3,5,-3>, <4,5,0>]]
   translate <-2, -2.5, 0>
   rotate <-90, -30, 0>
   uv_steps 32, 32
   shiny_red
   }
```

The preceeding patch was both non-uniform and rational. If you don’t want to bother declaring the knot vector you can simply omit it. This will result in a open uniform B-Spline patch. Most of the time the non-uniform knot vectors are unnecessary and can be safely omitted. The preceeding declaration with uniform knot vectors, and non-rational vertices could then be declared as:

```
object {
   nurb 4, 6, 4, 5,
    [[< 0, 0, 0>, < 1, 0, 3>, < 2, 0,-3>, < 3, 0, 3>, < 4, 0, 0>],
     [< 0, 1, 0>, < 1, 1, 0>, < 2, 1, 0>, < 3, 1, 0>, < 4, 1, 0>],
     [< 0, 2, 0>, < 1, 2, 0>, < 2, 2, 5>, < 3, 2, 0>, < 4, 2, 0>],
     [< 0, 3, 0>, < 1, 3, 0>, < 2, 3, 5>, < 3, 3, 0>, < 4, 3, 0>],
     [< 0, 4, 0>, < 1, 4, 0>, < 2, 4, 0>, < 3, 4, 0>, < 4, 4, 0>],
     [< 0, 5, 0>, < 1, 5,-3>, < 2, 5, 3>, < 3, 5,-3>, < 4, 5, 0>]]
   translate <-2, -2.5, 0>
   rotate <-90, -30, 0>
   uv_steps 32, 32
   shiny_red
}
```

Note that internally NURBS are stored as triangles. This can result in a high memory usage for a finely diced NURB (uv_steps large).

Sample files: nurb0.pi, nurb1.pi

2.3.2.13 Parabola

A parabola is defined in terms of a bottom point, a top point, and its radius at the top.

The format of the declaration is:

```
parabola <x0, y0, z0>, <x1, y1, z1>, r
```

The vector <x0,y0,z0> defines the top of the parabola - the part that comes to a point. The vector <x1,y1,z1> defines the bottom of the parabola, the width of the parabola at this point is r.

An example of a parabola declaration is:

```
object {
   parabola <0, 6, 0>, <0, 0, 0>, 3
   translate <16, 0, 16>
   steel_blue
   }
```

This is sort of like a salt shaker shape with a rounded top and the base on the x-z plane.

2.3.2.14 Parametric surface

A parametric surface allows the creation of surfaces as a mesh of triangles.
By defining the vertices of the mesh in terms of functions of u and v, Polyray will automatically create the entire surface. The smoothness of the surface is determined by the number of steps allowed for u and v.

The mesh defaults to 0 <= u <= 1, and 0 <= v <= 1. By explicitly defining the uv_bounds for the surface it is possible to create only the desired parts of the surface.

The format of the declaration is:

```
parametric <fx(u,v), fy(u,v), fz(u,v)>
```

For example, the following declarations could be used to create a torus:

```
define r0 1.25
define r1 0.5

define torux (r0 + r1 * cos(v)) * cos(u)
define toruy (r0 + r1 * cos(v)) * sin(u)
define toruz r1 * sin(v)

object {
   parametric <torux,toruy,toruz>
   rotate <-20, 0, 0>
   shiny_red
   uv_bounds 0, 2*pi, 0, 2*pi
   uv_steps 16, 8
   }
```

2.3.2.15 Polygon

Although polygons are not very interesting mathematically, there are many sorts of objects that are much easier to represent with polygons. Polyray assumes that all polygons are closed and automatically adds a side from the last vertex to the first vertex.

The format of the declaration is:

```
polygon total_vertices,
   <vert1.x,vert1.y,vert1.z>
   [, <vert2.x, vert2.y, vert2.z>]
   [, etc. for total_vertices vertices]
```

As with the sphere, note the comma separating each vertex of the polygon.

I use polygons as a floor in a lot of images. They are a little slower than the corresponding plane, but for scan conversion they are a lot easier to handle. An example of a checkered floor made from a polygon is:

```
object {
   polygon 4, <-20,0,-20>, <-20,0,20>, <20,0,20>, <20,0,-20>
   texture {
      checker matte_white, matte_black
      translate <0, -0.1, 0>
      scale <2, 1, 2>
      }
   }
```

Sample file: polygon.pi

2.3.2.16 Polynomial surface

The format of the declaration is:

```
polynomial f(x,y,z)
```

The function f(x,y,z) must be a simple polynomial, i.e. x^2+y^2+z^2-1.0 is the definition of a sphere of radius 1 centered at (0,0,0).

For quartic (4th order) equations, there are three ways that Polyray can use to solve for roots. By specifying which one is desired, it is possible to tune for quality or speed. The method of Ferrari is the fastest, but also the most numerically unstable. By default the method of Vieta is used. Sturm sequences (which are the slowest) should be used where the highest quality is desired.

The declaration of which root solver to use takes one of the forms:

```
root_solver Ferrari
root_solver Vieta
root_solver Sturm
```

(Capitalization is important - these are proper nouns after all.)

Note: due to unavoidable numerical inaccuracies, not all polynomial surfaces will render correctly from all directions. Please note that root finding for high-degree polynomials is inherently ill-conditioned. This is a mathematical property, not an implementation limitation. Wilkinson’s polynomial (1959) is the classic demonstra-

tion that roots of degree-4+ polynomials are exquisitely sensitive to coefficient perturbations.

The following example, taken from devil.pi defines a quartic polynomial. The use of the CSG clipping object is to trim uninteresting parts of the surface.
The bounding box declaration helps the scan conversion routines figure out where to look for the surface.

```
// Variant of a devil's curve in 3-space.  This figure has a top
// and bottom part that are very similar to a hyperboloid of one
// sheet, however the central region is pinched in the middle
// leaving two teardrop shaped holes.
object {
   object { polynomial x^4 + 2*x^2*z^2 - 0.36*x^2 - y^4 +
                       0.25*y^2 + z^4
            root_solver Ferrari }
   & object { box <-2, -2, -0.5>, <2, 2, 0.5> }
   bounding_box <-2, -2, -0.5>, <2, 2, 0.5>
   rotate <10, 20, 0>
   translate <0, 3, -10>
   shiny_red
   }
```

Note: as the order of the polynomial goes up, the numerical accuracy required to render the surface correctly also goes up. One problem that starts to rear its ugly head starting at around 3rd to 4th order equations is a problem with determining shadows correctly. The result is black spots (a.k.a shadow acne) on the surface. You can ease this problem to a certain extent by making the value of shadow_tolerance larger. For 4th and higher equations, you will want to use a value of at least 0.05, rather than the default 0.001.

Sample file: torus.pi, many others

2.3.2.17 Raw objects

Raw files are either bare bones ASCII files that describe triangles or files in WaveFront .obj format. RAW ASCII files typically contain the xyz coordinates of each of the three coordinates of a triangle, one set (nine values) per line. Sometimes these raw files will contain additional information such as normal values for the vertices or texture names to use for the triangles.

The format of the primitive is:

```
raw "rawfile" [, smooth_angle]
```

You can read an ASCII raw file containing lines with the following formats:

1. 9 vertices per line
2. 9 vertices, 9 normal values per line
3. 9 vertices, 9 normal values, 6 u/v values per line
4. 9 vertices per line followed by a texture name
5. A texture name alone, followed by lines with any of the formats in 1 through 4 above.

The WaveFront .obj files should all be parsed properly (if they meet the spec). However Polyray currently doesn't understand the following things that can be in a .obj file: smoothing groups and NURBS.

ASCII raw files aren't generally readable by a human. For example, the lines below are taken from a raw file:

```
2 -5e-08 -2e-07 1.848 0.7654 -1.848e-07 1.707 0.7654 -0.7071
2 -5e-08 -2e-07 1.707 0.7654 -0.7071 1.848 -5e-08 -0.7654
1.848 0.7654 -1.848e-07 1.414 1.414 -1.414e-07 1.307 1.414 -0.5412
1.848 0.7654 -1.848e-07 1.307 1.414 -0.5412 1.707 0.7654 -0.7071
1.414 1.414 -1.414e-07 0.7654 1.848 -7.654e-08 0.7071 1.848 -0.2929
1.414 1.414 -1.414e-07 0.7071 1.848 -0.2929 1.307 1.414 -0.5412
0.7654 1.848 -7.654e-08 1e-07 2 -1e-14 9.239e-08 2 -3.827e-08
0.7654 1.848 -7.654e-08 9.239e-08 2 -3.827e-08 0.7071 1.848 -0.2929
```

The file defines a single Polyray object. Note that the memory devoted to the raw object is only allocated once. This means that you can declare a raw object and make many instantiations of it without using any more memory than the first. This is very different than what you would get if you use RAW2POV.
In that case, each instantiation would hold a complete copy of the declared object, using up as much memory for a copy as for the original. (It's a problem with Polyray, not with RAW2POV.)

If a texture name appears (either at the end of a line, or by itself) then that texture will be used for the triangle, regardless of any other declarations that may appear in the data file. If you want to use different textures for the raw object then you should avoid using texture names in the raw data file.

For example, the following file demonstrates how to declare and use a raw skull:

```
viewpoint {
   from <0,0,-25>
   at <0,0,0>
   angle 25
   }
background white
light <-10,3, -20>
include "colors.inc"

define skull
object { raw "skull.raw" rotate <90, -30, 180> translate <0, 3, 0> }

skull { shiny_red }
skull { shiny_coral translate < 4, 4, 5> }
skull { shiny_green translate <-4, 4, 5> }
skull { metallic_magenta translate < 4,-4, 5> }
skull { metallic_orange translate <-4,-4, 5> }
```

The file “skull.raw” contains around a thousand triangles, one per line.

If an angle follows the raw file name, then all triangles that are next to each other and that form an angle less than the smoothing angle will have their normals averaged together. The smoothing angle must be between 0 and 180. At 0 there will be no smoothing and at 180 every normal will be smoothed. For example the following declaration will produce the skull with smoothing for triangles that are within 30 degrees of each other.

```
object { raw "skull.raw", 30 }
```

Note that the angles are measured by looking at the face normals of the triangles. This means that you need to have a consistent ordering of the vertices around each triangle. If you don’t have all vertices in the same order (e.g., clockwise) then you may have some smoothing errors.

2.3.2.18 Spheres

Spheres are the simplest 3D object to render and a sphere primitive enjoys a speed advantage over most other primitives.

The format of the declaration is:

```
 sphere <center.x, center.y, center.z>, radius
```

Note the comma after the center vector, it really is necessary.

The basic benchmark file is a single sphere, illuminated by a single light.
The definition of the sphere is:

```
object {
   sphere <0, 0, 0>, 2
   shiny_red
   }
```

Sample file: sphere.pi

2.3.2.19 Superquadric

The format of the primitive is:

```
superq n, e
```

Where “n” is the coefficient for shaping in the up/down direction and “e” is the shaping cofficient for the east/west (around) direction. The values for n and e MUST be larger than 0, and in fact the shape will probably fail if you use values lower than around 0.1.

The formula for the superquadric is:

$(|x|^{2/e} + |y|^{2/e})^{e/n} + |z|^{n} - 1.0 = 0$

(Bet that makes a lot of sense..) For example, the following object is a pinchy shape:

```
object { superq 3, 3 }
```

Where the one below is a cylinder with rounded edges:

```
object { superq 0.2, 1 }
```

These are nice values to use:

Shape	n	e
Cylinder	0.1	1
Rounded Box	0.1	0.1
Pillow	1	0.1
Sphere	1	1
Double Cone	2	1
Octahedron	2	2
Pinchy	3	3

2.3.2.20 Sweep surface

A sweep surface, also referred to as an extruded surface, is a polygon that has been swept along a given direction. It can be used to make multi-sided beams, or to create ribbon-like objects.

The format of the declaration is:

```
sweep type, direction, total_vertices,
   <vert1.x,vert1.y,vert1.z>
   [, <vert2.x, vert2.y, vert2.z>]
   [, etc. for total_vertices vertices]
```

The value of type is 1, 2, or 3. If the value is 1, then the surface will be a set of connected squares. If the value is 2, then the surface will be represented by a spline that approximates the line segments that were given.
If the value is 3, then the z component of each vertex is used to determine if it (the vertex) is on or off the curve (non-zero z for off-curve).

The value of the vector direction is used to change the orientation of the sweep. For a sweep surface that is extruded straight up and down the y-axis, use <0, 1, 0> for direction. The size of the vector direction will also affect the amount of extrusion (e.g., if |direction| = 2, then the extrusion will be two units in that direction).

An example of a sweep surface is:

```
// Sweep made from connected quadratic splines.
object {
   sweep 2, <0, 2, 0>, 16,
      <0, 0>, <0, 1>, <-1, 1>, <-1, -1>, <2, -1>, <2, 3>,
      <-4, 3>, <-4, -4>, <4, -4>, <4, -11>, <-2, -11>,
      <-2, -7>, <2, -7>, <2, -9>, <0, -9>, <0, -8>
   translate <0, 0, -4>
   scale <1, 0.5, 1>
   rotate <0,-45, 0>
   translate <10, 0, -18>
   shiny_yellow
   }
```

Sample file: sweep1.pi, sweep2.pi

Note: CSG will really only work correctly if you close the sweep - that is make the end point of the sweep the same as the start point. Sweeps, unlike polygons are not automatically closed by Polyray.

See the description of glyphs for a more general swept surface.

2.3.2.21 Torus

The torus primitive is a doughnut shaped surface that is defined by a center point, the distance from the center point to the middle of the ring of the doughnut, the radius of the ring of the doughnut, and the orientation of the surface.

The format of the declaration is:

```
torus r0, r1, <center.x, center.y, center.z>,
              <dir.x, dir.y, dir.z>
```

As an example, a torus that has major radius 1, minor radius 0.4, and is oriented so that the ring lies in the x-z plane would be declared as:

```
object {
   torus 1, 0.4, <0, 0, 0>, <0, 1, 0>
   shiny_red
   }
```

Note: since a torus is a 4th order polynomial, it is possible to specify which quartic root solver to use. See section 2.3.2.15 for a description of the root_solver statement.

Sample file: torus.pi

2.3.2.22 Triangular patches

A triangular patch is defined by a set of vertices and their normals. When calculating shading information for a triangular patch, the normal information is used to interpolate the final normal from the intersection point to produce a smooth shaded triangle.

The format of the declaration is:

```
patch <x0,y0,z0>, <nx0,ny0,nz0>,  [UV u0, v0,]
      <x1,y1,z1>, <nx1,ny1,nz1>,  [UV u1, v1,]
      <x2,y2,z2>, <nx2,ny2,nz2> [, UV u2, v2]
```

The vertices and normals are required for each of the three corners of the patch. The u/v coordinates are optional. If they are omitted, then they will be set to the following values:

```
u0 = 0, v0 = 0
u1 = 1, v1 = 0
u2 = 0, v1 = 1
```

Smooth patch data is usually generated as the output of another program.

2.3.3 Constructive Solid Geometry (CSG)

Objects can be defined in terms of the union, intersection, and inverse of other objects. The operations and the symbols used are:

```
csgexper + csgexper  - Union
csgexper ^ csgexper  - Merge (union with internal surfaces removed)
csgexper * csgexper  - Intersection
csgexper - csgexper  - Difference
csgexper & csgexper  - Clip the first object by the second
~csgexper            - Inverse
(csgexper)           - Parenthesised expression
```

Note that intersection and difference require a clear inside and outside. Not all primitives have well defined sides. Those that do are:

Spheres, Boxes, Glyphs, Polynomials, Blobs, Tori, and Functions.

Other surfaces that do not always have a clear inside/outside, but work reasonably well in CSG intersections are:

```
Cylinders, Cones, Discs, Height Fields, Lathes, Parabola, Polygons,
and Sweeps.
```

Using Cylinders, Cones, and Parabolas works correctly, but the open ends of these surfaces will also be open in the resulting CSG. To close them off you can use a disc shape.

Using Discs, and Polygons in a CSG is really the same as doing a CSG with the plane that they lie in. If fact, a large disc is an excellent choice for clipping or intersecting an object, as the inside/outside test is very fast.

Lathes, and Sweeps use Jordan’s rule to determine if a point is inside. This means that given a test point, if a line from that point to infinity crosses the surface an odd number of times, then the point is inside. The net result is that if the lathe (or sweep) is not closed, then you may get odd results in a CSG intersection (or difference).

CSG involving height fields only works within the bounds of the field.

As an example, the following object is a sphere of radius 1 with a hole of radius 0.5 through the middle:

```
define cylinder_z object { cylinder <0,0,-1.1>, <0,0,1.1>, 0.5 }
define unit_sphere object { sphere <0, 0, 0>, 1 }

// Define a CSG shape by deleting a cylinder from a sphere
object {
   unit_sphere - cylinder_z
   shiny_red
   }
```

Sample files: csg1.pi, csg2.pi, csg3.pi, roman.pi, lens.pi, polytope.pi

2.3.4 Gridded objects

A gridded object is a way to compactly represent a rectangular arrangement of objects by using an image map. Each object is placed within a 1x1 cube that has its lower left corner at the location <i, 0, j> and its upper right corner at <i+1, 1, j+1>. The color index of each pixel in the image map is used to determine which of a set of objects will be placed at the corresponding position in space.

The gridded object is much faster to render than the corresponding layout of objects. The major drawback is that every object must be scaled and translated to completely fit into a 1x1x1 cube that has corners at <0,0,0> and <1,1,1>.

The size of the entire grid is determined by the number of pixels in the image. A 16x32 image would go from 0 to 16 along the x-axis and the last row would range from 0 to 16 at 31 units in z out from the x-axis.

The format of the declaration is:

```
gridded "image.tga",
   object1
   object2
   object3
   ...
```

An example of how a gridded object is declared is:

```
define tiny_sphere object { sphere <0.5, 0.4, 0.5>, 0.4 }
define pointy_cone object { cone <0.5, 0.5, 0.5>, 0.4,
                                 <0.5, 1, 0.5>, 0 }

object {
   gridded "grdimg0.tga",
      tiny_sphere { shiny_coral }
```

```
      tiny_sphere { shiny_red }
      pointy_cone { shiny_green }
   translate <-10, 0, -10>
   rotate <0, 210, 0>
   }
```

In the image grdimg0.tga, there are a total of 3 colors used, every pixel that uses color index 0 will generate a shiny coral colored sphere, every pixel that uses index will generate a red sphere, every pixel that uses index 2 will generate a green cone, and every other color index used in the image will leave the corresponding space empty.

The normal image format for a gridded object is either grayscale or color mapped. To determine which object will be used, the value of the pixel itself in the grayscale image and the color index is used in the mapped image. If a 16 bit image is used, then the second byte of the color is used. If a 24 or 32 bit image is used, then the value of the red component is used. This limits the # of objects for all images to
256.

A color JPEG is treated the same as a 24 bit Targa (unlikely to be useful, due to the lossy nature of JPEG).

Sample files: grid0.pi, river.pi

2.3.5 Particle Systems

There are two distinct pieces of information that are used by Polyray to do particles. The first is a particle declaration, this is the generator. The second is the particle itself. It is important to retain the distinction, you generally only want to have one particle generator, but you may want that generator to produce many particles.

The form of the declaration for a particle generator is:

```
particle {
   object "name"
   position vector
   velocity vector
   acceleration vector
   birth float
   death float
   count float
```

```
   avoid float
   }
```

The wrapper for the particle must be the name of an object appearing in a define statement. If there is no object with that name, Polyray will write an error message and abort. Everything else is optional, but if you don’t set either the velocity or acceleration then the object will just sit at the origin.

The default values for each component are:

```
position     - <0, 0, 0>
velocity     - <0, 0, 0>
acceleration - <0, 0, 0>
birth        - frame == start_frame
death        - false (never dies)
count        - 1
avoid        - false (doesn't bounce)
```

As an example, the following declaration makes a starburst of 50 spheres.
Note the conditional before the particle declaration. You almost always want to do this, otherwise you will create a new particle generator at every frame of the animation. Since the default birth condition is to generate particles only on the first frame, the only side effect here would be to suck up more memory every frame to retain the particle generator definition.

```
frame_time 0.05

define gravity -1

// Star burst
if (frame == start_frame)
particle {
   position <0, 5, 0>
   velocity brownian(<0, 0, 0>, <1, 1, 1>)
   acceleration gravity
   object "bsphere"
   count 50
   }
```

The value in the velocity declaration generates a random vector that ranges from <–1, –1, –1> to <1, 1, 1>. Each particle of the 50 is given a different initial velocity, which is what gives the bursting effect.

An additional declaration, frame_time xx, has been added specifically for tuning particle animations. This declaration determines how much time passes for every frame. Each particle starts with an age of 0, after each frame it's age is incremented by the value xx in the frame_time declaration.

Additionally the position and velocity of the particle is updated after every frame according to the formula:

```
V = V + frame_time * A
P = P + frame_time * V
```

The status of a particle is set by making use of some of the standard Polyray variables. The names and meanings are:

```
P   - Current location of the particle as a vector
x   - X location of the particle (or P[0])
y   - Y location of the particle (or P[1])
z   - Z location of the particle (or P[2])
I   - Current velocity of the particle as a vector
u   - Age of the particle (frame_time * elapsed frames since birth)
```

These values are used in two situations, when checking the death condition of a particle and when calculating the acceleration of the particle.

If an avoid statement is given then before every frame the position of the particle is checked to see if it hits any objects in the scene (non-particle objects). If so, then the particle will bounce off the object.

Note: see section 1.6.1.2 for information on lazy evaluation of expressions.
This is necessary for some particle systems to behave correctly.

Sample files: part1.pi, part2.pi, part3.pi, part6.pi

2.3.6 Finding bounds of objects

It is possible to have Polyray figure out the bounds of an object for use in later operations. For example, you may have a glyph object and you want to center it. To do this, the functions min() and max() have been enhanced to understand objects. Unlike their normal use of returning the smallest/largest of two numbers, when you give them the name of a defined object in double quotes (e.g., "sphere1") then they will return the lower left corner or upper right corner of the bounding box of the object.

For example, if you have a complicated glyph (like the character ‘a’ using a serif font):

```
define char_a
object { // Char: 'a'
   glyph 2
      contour 35, <1.119434, 0.000000, 0>, <1.105273, 0.043945, 1>,
      <1.105273, 0.075684, 0>, <1.045703, 0.016602, 1>,
...
```

To determine the size of this character along each axis you would simply subtract the min value from the max value:

```
define glypha_width max("char_a") - min("char_a")
```

If you just wanted the width along the x-axis, then you would look at the first coordinate of the vector above:

```
define glypha_xwidth (max("char_a") - min("char_a"))[0]
```

Using the width of the character, we can now accurately place several of these next to each other:

```
define five_a
object {
   char_a +
   char_a { translate glypha_width } +
   char_a { translate 2*glypha_width } +
   char_a { translate 3*glypha_width } +
   char_a { translate 4*glypha_width }
   }
```

The center of an object can also be determined very easily. By figuring the average of the min and max, you get the center of the bounding box. The example below shows how you can find the center of an object and then translate so that the center of the object is right at the origin (<0,0,0>).

```
define five_a_width (min("five_a") + max("five_a"))/2

// Instantiate the character a, centered at the origin
five_a { translate -five_a_width }
```

2.4 Color and lighting

The color space used in polyray is RGB, with values of each component specified as a value from 0 -> 1. The way the color and shading of surfaces is specified is described in the following sections.

RGB colors are defined as either a three component vector, such as <0.6, 0.196078, 0.8>, or as one of the X11R3 named colors (which for the value given is DarkOrchid). One of these days when I feel like typing and not thinking (or if I find them on line), I'll put in the X11R4 colors.

The coloring of objects is determined by the interaction of lights, the shape of the surface it is striking, and the characteristics of the surface itself.

2.4.1 Light sources

Light sources are one of: simple positional light sources, spot lights, or textured lights. None of these lights have any physical size. The lights do not appear in the scene, only the effects of the lights.

The keyword "no_shadow" can be added to all of the supported light types to prevent shadows from being cast by that light. The no_shadow comes right after the keywords: light, spot_light, and directional_light. It may appear anywhere within the declaration of textured_light and depthmapped_light.

For example the following declaration is a point light at <0, 10, 0> that doesn't cast a shadow:

```
light no_shadow <0, 10, 0>
```

Any light can be turned into an object. This is really useful for adding lights to CSG unions. The format is:

```
object {
   ... standard light declaration ...
   }
```

Yes, you can add scale/rotate/translate declarations. They will do the sensible things to the light. For example:

```
   object {
      object {
         light white, <0, 0, 0>
         translate <20, 20, -40>
         } +
      object {
         sphere <0, 0, 0>
         shiny_red
         }
```

```
rotate <0, 30, 0>
}
```

2.4.1.1 Positional Lights

A positional light is defined by its RGB color and its XYZ position.

The formats of the declaration are:

```
light color, location
light location
```

The second declaration will use white as the color.

2.4.1.2 Spot Lights

The formats of the declaration are:

```
spot_light color, location, pointed_at, tightness, angle, falloff
spot_light location, pointed_at
```

The vector location defines the position of the spot light, the vector pointed_at defines the point at which the spot light is directed. The optional components are:

```
color        - The color of the spotlight
tightness    - The power function used to determine the shape of
               the hot spot
angle        - The angle (in degrees) of the full effect of the
               spot light
falloff      - A larger angle at which the amount of light falls
               to nothing
```

A sample declaration is:

```
spot_light white, <10,10,0>, <3,0,0>, 3, 5, 20
```

Sample file: spot0.pi

2.4.1.3 Directional lights

The directional light means just that - light coming from some direction.

```
directional_light color, direction
 directional_light direction
```

An example would be: directional_light <2, 3, −4>, giving a white light coming from the right, above, and behind the origin.

2.4.1.4.1 Area Lights

Two forms of area lights: spherical and planar.

2.4.1.4.1.1 Spherical Area Lights

By adding a sphere declaration to a textured_light, it is turned into an area light. The current implementation is a bit rough for long and narrow shadows, but in general gives very good results. A typical declaration of an area light is:

```
textured_light {
   color white
   sphere <8, 10, 0>, 1
   }
```

Sample file: spoty.pi

2.4.1.4.1.2 Planar Area Lights

A textured light that has a polygon definition is treated as a flat area light. The starting size is a box in the x-z plane, from the origin to <1, 0, 1>. You rotate/scale/translate it into position from there.
The declaration of the polygon has the following parameters:

```
textured_light {
   color ...
   polygon xres, zres, adaptive_depth, jitter_amount
   ...
   }
```

The rest of the entries are the same as for normal textured lights - you can put a standard color or a formula for the color, you can transform it, all that sort of stuff.

The values of xres and zres determine the minimum number of times the light will be sampled. If adaptive_depth is greater than 0 then the polygon will be subdivided using a method identical to the way antialiasing works. Jitter determines how much each shadow ray is wiggled inside each piece of the light (a value of 0.25 works pretty well).

For example, this creates an area light centered at the location <10, 8, 0>, with sides a single unit, and pointed down the x-axis.

```
textured_light {
   color white
   polygon 2, 2, 1, 0.05
   translate <-0.5, 0, -0.5>
   rotate <0, 0, 90>
   translate <10,8,0>
   }
```

2.4.1.4 Textured lights

Textured lights are an enhancement of point lights that add: a function (including image maps) to describe the intensity & color of the light in each direction, transformations, and size. The format of the declaration is:

```
textured_light {
   color float
   [sphere center, radius]
   [translate/rotate/scale]
   }
```

Any color expression is allowed for the textured light, and is evaluated at run time.

A rotating slide projector light from the data file ilight.pi is shown below:

```
define block_environ
environment("one.tga", "two.tga", "three.tga",
            "four.tga", "five.tga", "six.tga")
textured_light {
   color environment_map(P, block_environ)
   rotate <frame*6, frame*3, 0>
   translate <0, 2, 0>
   }
```

Sample files: environ.pi, ilight.pi

2.4.1.5 Depth Mapped Lights

Depth mapped lights are very similar to spotlights, in the sense that they point from one location and at another location. The primary use for this light type is for doing shadowing in scan converted scenes. The format of their declaration is:

```
depthmapped_light {
   [ angle float ]
   [ aspect float ]
   [ at vector ]
   [ color expression ]
   [ depth "depthfile.tga" ]
   [ from vector ]
   [ hither float ]
   [ up vector ]
   }
```

You may notice that the format of the declaration is very similar to the viewpoint declaration. This is intentional, as you will usually

generate the depth information for depthfile.tga as the output of a run of Polyray. To support output of depth information, a new statements was added to the viewpoint declaration, image_format.

A viewpoint declaration that will output a depth file would have the form:

```
viewpoint {
   from [ location of depth mapped light ]
   at   [ location the light is pointed at ]

   image_format 1
   }
```

Where the final statement tells Polyray to output depth information instead of color information. Note that if the value in the image_format statement is 0, then normal rendering will occur.

If a hither declaration is used, then the value given is used as a bias to help prevent self shadowing. The default value for this bias is the value of shadow_tolerance in polyray.ini.

Sample File: room1.pi, depmap.pi

2.4.2 Background color

The background color is the one used if the current ray does not strike any objects. The color can be any vector expression, although is usually a simple RGB color value.

The format of the declaration is:

```
background <R,G,B>
```

or

```
background color
```

If no background color is set black will be used.

An interesting trick that can be performed with the background is to use an image map as the background color (it is also a way to translate from one Targa format to another). The way this can be done is:

background planar_imagemap(image(“test1.tga”, P)

The background also affects the opacity channel in 16 and 32 bit Targa output images. If any background contributes directly to the pixel (not as a result of reflection or refraction), then the

attribute bit is cleared in a 16 bit color and the attribute byte is set to the percentage of the pixel that was covered by the background in a 32 bit color.

In order to extend the flexibility of the background, the meanings of various runtime variables are set uniquely.

```
u,x - How far across the output image (from 0 at left to 1 at
      right)
v,z - How far down the output image (from 0 at bottom to 1 at top)
W   - <0,0,0>
P   - Same as <x, 0, z>
N   - Direction of the current ray
I   - <0,0,0>
U   - Same as <u, v, w>
w   - Level of recursion (0 for eye rays, higher for reflected
      and refracted rays)
```

As an example, suppose you wanted to have an image appear in the background, but you didn't want to have the image appear in any reflections. Then you could define the background with the following expression:

```
background (w == 0 ? planar_imagemap(img1, P) : black)
```

If you wanted to have one image in the background, and another that appears in reflections (or refracted rays), then you could use the following expression:

```
background (w == 0 ? planar_imagemap(img1, P)
                   : spherical_imagemap(img2, N))
```

The previous background expression fits img1 exactly into the output image and maps img2 completely around the entire scene. This might be useful if you have a map of stars that you want to appear in reflections, but still want to have an image as the background.

2.4.2.1 Global Haze (fog)

The global haze is a color that is added based on how far the ray traveled before hitting the surface. The format of the expression is:

```
haze coeff, starting_distance, color
```

The color you use should almost always be the same as the background color. The only time it would be different is if you are trying to put haze into a valley, with a clear sky above (this is a tough

trick, but looks nice). A
example would be:

```
haze 0.8, 3, midnight_blue
```

The value of the coeff ranges from 0 to 1, with values closer to 0 causing the haze to thicken, and values closer to 1 causing the haze to thin out. I know it seems backwards, but it is working and I don’t want to break anything.

2.4.3 Textures

Polyray supports a few simple procedural textures: a standard shading model, a checker texture, a hexagon texture, and a general purpose (3D noise based) texture. In addition, a very flexible (although slower) functional texture is supported. Individual textures can be combined in various ways to create new ones. Texture types that help to combine other textures include: layered textures, indexed textures, and summed textures. (See section 2.4.3.3 through section 2.4.3.5).

The general syntax of a texture is:

```
texture { [texture declaration] }
```

or

```
texture_sym
```

Where texture_sym is a previously defined texture declaration.

2.4.3.1.1.1 Ambient light

Ambient lighting is the light given off by the surface itself. This will be a constant amount, independent of any lights that may be in the scene.

The format of the declaration is:

```
ambient color, scale
ambient scale
```

As always, color indicates either an RGB triple like <1.0,0.7,0.9>, or a named color. scale gives the amount of contribution that ambient gives to the overall amount light coming from the pixel. The scale values should lie in the range 0.0 -> 1.0

2.4.3.1.1.2 Diffuse light

Diffuse lighting is the light given off by the surface under stimulation by a light source. The intensity of the diffuse light is directly proportional to the angle of the surface with respect to the light.

The format of the declaration is:

```
diffuse color, scale
diffuse scale
```

The only information used for diffuse calculations are the angle of incidence of the light on the surface and the value of the brilliance component. The format for brilliance is:

```
brilliance scale
```

Brilliance is sometimes used for a slightly more metallic look. While it has no particular physical significance, by setting it to a value larger than 1.0 the diffuse highlight will be smaller. Compare the shading of the two spheres listed below:

```
object {
   sphere <0, 0, 0>, 1
   texture { shiny { color gold } }
   translate <-1, 0, 0>
   }

object {
   sphere <0, 0, 0>, 1
   texture { shiny { color gold specular 0.6 brilliance 2 } }
   translate <1, 0, 0>
   }
```

2.4.3.1.1.3 Specular highlights

The format of the declaration is:

```
specular color, scale
specular scale
```

The means of calculating specular highlights is by default the Phong model.
Other models are selected through the Microfacet distribution declaration.

2.4.3.1.1.4 Reflected light

Reflected light is the color of whatever lies in the reflected direction as calculated by the relationship of the view angle and the normal to the surface.

The format of the declaration is:

```
reflection scale
reflection color, scale
```

Typically, only the scale factor is included in the reflection declaration, this corresponds to all colors being reflected with intensity proportional to the scale. A color filter is allowed in the reflection definition, and this allows the modification of the color being reflected (I'm not sure if this is useful, but I included it anyway).

2.4.3.1.1.5 Transmitted light

Transmitted light is the color of whatever lies in the refracted direction as calculated by the relationship of the view angle, the normal to the surface, and the index of refraction of the material.

The format of the declaration is:

```
transmission scale, ior
transmission color, scale, ior
```

Typically, only the scale factor is included in the transmitted declaration, this corresponds to all colors being transmitted with intensity proportional to the scale. A color filter is allowed in the transmitted definition, and this allows the modification of the color being transmitted by making the transmission filter different from the color of the surface itself.

It is possible to have surfaces with colors very different than the one closest to the eye become apparent. (See gsphere.pi for an example, a red sphere is in the foreground, a green sphere and a blue sphere behind. The specular highlights of the red sphere go to yellow, and blue light is transmitted through the red sphere.)

A more complex file is lens.pi in which two convex lenses are lined up in front of the viewer. The magnified view of part of a grid of colored balls is very apparent in the front lens.

2.4.3.1.1.6 Microfacet distribution

The microfacet distribution is a function that determines how the specular highlighting is calculated for an object.

The format of the declaration is:

```
microfacet Distribution_name falloff_angle
microfacet falloff_angle
```

The distribution name is one of: Blinn, Cook, Gaussian, Phong, Reitz. The falloff angle is the angle at which the specular highlight falls to 50% of its maximum intensity. (The smaller the falloff angle, the sharper the highlight.) If a microfacet name is not given, then the Phong model is used.

The falloff angle must be specified in degrees, with values in the range 0 to 45. The falloff angle corresponds to the roughness of the surface, the smaller the angle, the smoother the surface.

Note: as stated before, look at the book by Hall. I have found falloff values of 5-10 degrees to give nice tight highlights. Using falloff angle may seem a bit backwards from other raytracers, which typically use the power of a cosine function to define highlight size. When using a power value, the higher the power, the smaller the highlight. Using angles seems a little tidier since the smaller the angle, the smaller the highlight.

2.4.3.1.1 Standard Shading Model

Unlike many other ray-tracers, surfaces in Polyray not have a single color that is used for all of the components of the shading model. Instead a number of characteristics of the surface must be defined (with a matte white being the default).

A surface declaration has the form:

```
surface {
   [ surface definitions ]
   }
```

For example, the following declaration is a red surface with a white highlight, corresponding to the often seen plastic texture:

```
define shiny_red
texture {
   surface {
      ambient red, 0.2
      diffuse red, 0.6
```

```
      specular white, 0.8
      microfacet Reitz 10
      }
   }
```

The allowed surface characteristics that can be defined are:

```
color               - Color if not given in another component
ambient             - Light given off by the surface
brilliance          - Tightness of diffuse reflections
diffuse             - Light reflected in all directions
specular            - Amount and color of specular highlights
reflection          - Reflectivity of the surface
transmission        - Amount and color of refracted light
microfacet          - Specular lighting model (see below)
```

The lighting equation used is (in somewhat simplified terms):

$L = ambient + diffuse^{brilliance} + specular + reflected + transmitted,$

or

$L = K_a + K_d * (l_1 + l_2 + \ldots) + K_s * (l_1 + l_2 + \ldots) + K_r + K_t$

Where l_1, l_2, … are the lights, K_a is the ambient term, K_d is the diffuse term, K_s is the specular term, K_r is the reflective term, and K_t is the transmitted (refractive) term. Each of these terms has a scale value and a filter value (the filter defaults to white/clear if unspecified).

See the file colors.inc for a number of declarations of surface characteristics, including: mirror, glass, shiny, and matte.

For lots of detail on lighting models, and the theory behind how color is used in computer generated images, run (don’t walk) down to your local computer bookstore and get:

```
Illumination and Color in Computer Generated Imagery
Roy Hall, 1989

Springer Verlag
```

Source code in the back of that book was the inspiration for the microfacet distribution models implemented for Polyray.

Note that you don’t really have to specify all of the color components if you don’t want to. If the color of a particular part of the surface declaration is not defined, then the value of the color component will be examined to see if it was declared. If so,

then that color will be used as the filter. As an example, the declaration above could also be written as:

```
define shiny_red
texture {
   surface {
      color red
      ambient 0.2
      diffuse 0.6
      specular white, 0.8
      microfacet Reitz 10
      }
   }
```

2.4.3.1.2 Checker

the checker texture has the form:

```
texture {
   checker texture1, texture2
   }
```

where texture1 and texture2 are texture declarations (or texture constants).

A standard sort of checkered plane can be defined with the following:

```
// Define a matte red surface
define matte_red
texture {
   surface {
      ambient red, 0.1
      diffuse red, 0.5
      }
   }

// Define a matte blue surface
define matte_blue
texture {
   surface {
      ambient blue, 0.2
      diffuse blue, 0.8
      }
   }

// Define a plane that has red and blue checkers
object {
   disc <0, 0.01, 0>, <0, 1, 0>, 5000
   texture {
      checker matte_red, matte_blue
```

```
      }
   }
```

For a sample file, look at spot0.pi. This file has a sphere with a red/blue checker, and a plane with a black/white checker.

2.4.3.1.3 Hexagon

the hexagon texture is oriented in the x-z plane, and has the form:

```
texture {
   hexagon texture1, texture2, texture3
   }
```

This texture produces a honeycomb tiling of the three textures in the declaration. Remember that this tiling is with respect to the x-z plane, if you want it on a vertical wall you will need to rotate the texture.

2.4.3.1.4 Noise surfaces

The complexity and speed of rendering of the noise surface type lies between the standard shading model and the special surfaces described below. It is an attempt to capture a large number of the standard 3D texturing operations in a single declaration.

A noise surface declaration has the form:

```
texture {
   noise surface {
      [ noise surface definition ]
      }
   }
```

The allowed surface characteristics that can be defined are:

```
color <r, g, b>          - Basic surface color (used if the noise
                           function generates a value not contained
                           in the color map)
ambient scale            - Amount of ambient contribution
diffuse scale            - Diffuse contribution
specular color, scale    - Amount and color of specular highlights,
                           if the color is not given then the body
                           color will be used.
reflection scale         - Reflectivity of the surface
transmission scale, ior  - Amount of refracted light
microfacet kind angle    - Specular lighting model (see the
                           description of a standard surface)

color_map(map_entries)   - Define the color map (see following
                           section on color map definitions for
```

```
                          further details)
   bump_scale float       - How much the bumps affect the normal
   frequency float        - Affects the wavelength of ripples and
                            waves
   phase float            - Affects the phase of ripples and waves
   lookup_fn index        - Selects a predefined lookup function
   normal index           - Selects a predefined normal modifier
   octaves float          - Number of octaves of noise to use
   position_fn index      - How the intersection point is used in
                            the generation of a noise texture
   position_scale float   - Amount of contribution of the position
                            value to the overall texture
   turbulence float       - Amount of contribution of the noise to
                            overall texture.
```

The way the final color of the texture is decided is by calculating a floating point value using the following general formula:

```
   index = lookup_fn(position_scale * position_fn +
                   turbulence * noise3d(P, octaves))
```

The index value that is calculated is then used to lookup a color from the color map. This final color is used for the ambient, diffuse, reflection and transmission filters. The functions that are currently available, with their corresponding indices are:

Position functions:

Index	Effect
1	x value in the object coordinate system
2	x value in the world coordinate system
3	Distance from the z axis
4	Distance from the origin
5	Angle from the x-axis (in the x-z plane, from 0 to 1)
default:	0.0

Lookup functions:

Index	Effect
1	sawtooth function, result from 0 -> 1
2	sin function, result from 0->1
3	ramp function, result from 0->1
default:	no modification made

Definitions of these function numbers that make sense are:

```
define position_plain       0
define position_objectx     1
define position_worldx      2
define position_cylindrical 3
define position_spherical   4
define position_radial      5

define lookup_plain    0
define lookup_sawtooth 1
define lookup_sin      2
define lookup_ramp     3
```

An example of a texture defined this way is a simple white marble:

```
define white_marble_texture
texture {
   noise surface {
      color white
      position_fn position_objectx
      lookup_fn lookup_sawtooth
      octaves 3
      turbulence 3
      ambient 0.2
      diffuse 0.8
      specular 0.3
      microfacet Reitz 5
      color_map(
         [0.0, 0.8, <1, 1, 1>, <0.6, 0.6, 0.6>]
         [0.8, 1.0, <0.6, 0.6, 0.6>, <0.1, 0.1, 0.1>])
      }
   }
```

In addition to coloration, the bumpiness of the surface can be affected by selecting a function to modify the normal. The currently supported normal modifier functions are:

Index	Effect
1	Make random bumps in the surface
2	Add ripples to the surface
3	Give the surface a dented appearance
default:	no change

Definitions that make sense are:

```
define default_normal 0
define bump_normal    1
define ripple_normal  2
define dented_normal  3
```

See also the file texture.txt for a little more explanation.

2.4.3.1 Procedural Textures

Procedural textures (i.e. checker, matte_white, shiny_red, …) are ones that are completely defined at the time the data file is read. Textures that are evaluated during rendering are described in section 2.4.3.2.

2.4.3.2.1.1 Transparency color and opacity

Special surfaces have a few special rules for how they use color and transparency. They are almost entirely focused on surfaces that have the form:

```
special surface {
   color color_map(...)[...]
   ambient ...
   ...
   }
```

If there is an alpha value in the color map, then the transmission scale will be set according to the alpha value. This will always happen, even if you put in a value for the transmission scale.

The transmission color will be set to the color that came out of the color_map only if there isn’t a transmission statement (or the transmission statement only has scale & ior values). If there is a transmission statement with a color, that that color will be used to filter light passing through the surface. The change from previous versions is that Polyray used to use white as the transmission color if there was an alpha value in the color_map.

Two new runtime variables have been added to ease texture design. The variable ‘color’ is set to the function in the color statement in the special surface. The variable ‘opacity’ is set to 1 - alpha (where the alpha value came from the color statement. This is useful if you want to adjust the diffuse (or specular) lighting based on transparency (like POV-Ray does in its textures). For example, this is a texture that has a color map that goes from white to red and from opaque to transparent, and diminishes the diffuse & specular:

```
texture {
   special surface {
      color color_map([0.0, 0.3, white, white]
                      [0.3, 1.0, white, 0, red, 1])
      diffuse 0.8 * opacity
      specular white, 0.4 * opacity
      }
   }
```

Note: use “white, 1” instead of “black, 1” to mean transparent.

Sample files: marble.pi

2.4.3.2.1.2 Using CMAPPER

ColorMapper used to be a good way to build color maps for layered textures is with ColorMapper, however since it is a 16-bit application it does not run under Windows anymore.

```
Written by : SoftTronics, Lutz + Kretzschmar
```

This is available as cmappov2.zip online. This program allows you to build color maps with varying colors and transparency values. The output of this program does have to be massaged a little bit to make it into a color map as Polyray understands it. In order to help with this process a Windows executable, makemap.exe has been included. To use this little program, you follow these steps:

1) run CMAPPER to create a color map in the standard output (not the POV-Ray output format).

2) run makemap on that file, giving a name for the new Polyray color map definition

3) Add this definition to your Polyray data file.

If you saved your map as foo.map, and you wanted to add this color map to the Polyray data file foo.inc, with the name of foox_map, you would then run makemap the following way:

```
makemap foo.map foox_map >> foo.inc
```

This makes the translation from CMAPPER format to Polyray format, and appends the output as, define foox_map color_map(…), to the file foo.inc.

2.4.3.2.1 Color maps

Color maps are generally used in noise textures and functional textures. They are a way of representing a spectrum of colors that blend from one into another. Each color is represented as RGB, with an optional alpha (transparency) value. The format of the declaration is:

```
color_map([low0, high0, <r0, g0, b0>, a0, <r1, g1, b1>, a1]
          [low1, high1, <r2, g2, b2>, a2, <r3, g3, b3>, a3]
          ...
          [lowx, highx, <rx, gx, bx>, ax, <ry, gy, by>, ay])
```

Note that there are no commas between entries in the color map even though commas are required within each entry. (This is a holdover to retain compatibility with earlier versions of Polyray.) If you don’t need any transparency in the color map, then the following declaration could be used:

```
color_map([low0, high0, <r0, g0, b0>, <r1, g1, b1>]
          [low1, high1, <r2, g2, b2>, <r3, g3, b3>]
          ...
          [lowx, highx, <rx, gx, bx>, <ry, gy, by>])
```

In this case, the alpha is set to 0 for all colors created by the color map.
Note that it is possible to mix colors with and without alpha value.

Note: If there is an alpha value in the color map, then the amount of alpha is used as the scale for the transmission component of the surface. To turn off this automatic transparency, use transmission 0.

2.4.3.2.2 Image maps

Projecting an image onto a surface is one of the most common texturing techniques. There are four types of projection supported: planar, cylindrical, spherical, and environment. Input files for use as image maps may be any valid Targa, PNG, or JPEG image.

The declaration of an image map is:

```
image("imfile.tga")
```

Typically, an image will be associated with a variable through a definition such as:

```
define myimage image("imfile.tga")
```

The image is projected onto a shape by means of a projection. The four types of projection are declared by:

```
planar_imagemap(image, coordinate [, repeat]),
cylindrical_imagemap(image, coordinate [, repeat]),
spherical_imagemap(image, coordinate)
environment_map(coordinate,
                environment("img1.tga", "img2.tga", "img3.tga",
                            "img4.tga", "img5.tga", "img6.tga"))
```

The planar projection maps the entire raster image into the coordinates 0 <= x <= 1, 0 <= z <= 1. The vector value given as coordinate is used to select a color by multiplying the x value by the number of columns, and the z value by the number of rows. The color appearing at that position in the raster will then be returned as the result. If a repeat value is given then entire surface, repeating between every integer Value of x and/or z.

The planar image map is also used for wrapping images based on u/v coordinates. By using the vector <u, 0, v>, the planar image map will automatically wrap the image around surfaces that have natural u/v coordinates (such as sphere, cylinder, torus, etc.).

The cylindrical projection wraps the image about a cylinder that has one end at the origin and the other at <0, 1, 0>. If a repeat value is given, then the image will be repeated along the y-axis, if none is given, then any part of the object that is not covered by the image will be given the color of pixel (0, 0).

The spherical projection wraps the image about an origin centered sphere. The top and bottom seam are folded into the north and south poles respectively.
The left and right edges are brought together on the positive x axis.

The environment map wraps six images around a point. This method is a standard way to fake reflections by wrapping the images that would be seen from a point inside an object around the object. A sample of this technique can be seen by rendering room1.pi (which generates the images) followed by rendering room0.pi (which wraps the images around a sphere).

Following are a couple of examples of objects and textures that make use of image maps:

```
define hivolt_image image("hivolt.tga")
define hivolt_tex
texture {
   special surface {
      color cylindrical_imagemap(hivolt_image, P, 1)
      ambient 0.9
      diffuse 0.1
      }
   scale <1, 2, 1>
   translate <0, -1, 0>
   }
object { cylinder <0, -1, 0>, <0, 1, 0>, 3 hivolt_tex }
```

and

```
define disc_image image("test.tga")
define disc_tex
texture {
   special surface {
      color planar_imagemap(disc_image, P)
      ambient 0.9
      diffuse 0.1
      }
   translate <-0.5, 0, -0.5>
   scale <7*4/3, 1, 7>
   rotate <90, 0, 0>
   }
object {
   disc <0, 0, 0>, <0, 0, 1>, 6
   u_steps 10
   disc_tex
   }
```

Note: If there is an alpha/opacity value in the image map, then the amount of opacity is used as the scale for the transmission component of the surface. To turn off this automatic transparency, use transmission
0.

Sample files: map0.pi, map1.pi, map2.pi

2.4.3.2.3 Bumpmaps

Bumpmaps are declared using the same sort of projections as image maps (excepting environment maps). The following are the valid declarations of bump maps:

```
planar_bumpmap(image, coordinate [, bump size]),
cylindrical_bumpmap(image, coordinate [, bump size]),
spherical_bumpmap(image, coordinate [, bump size])
```

Instead of an optional repeat argument, bumpmaps have an optional bump size argument. If this argument is left out, then the bump size is set to one.
Note that negative bump values are allowed and cause the bumps to appear to project the other way.

A bump map starts by interpreting pixels as if they were depths (see section 3.2.2). It then uses adjacent pixels to determine the slope (rate of change) of depths at that point. The rate of change along two different directions then defines how the normal to

Any image can be used, the interpretation of depth changes between pixels will be based on the depth file formats listed in section 3.2.2. The following declarations show how a bump map can be applied to objects:

```
include "colors.inc"

define tile_bumps image("tile1.tga")

define bumpmap_red1
texture {
   special shiny {
      color red
      normal planar_bumpmap(tile_bumps, <8*u, 0, 6*v>, 1)
      }
   }
object {
     object { torus 2, 0.75, <0, -1.25, 0>, <0, 1, 0> }
   + object { cone <0, -2, 0>, 1, <0, 3, 0>, 0 }
   + object { sphere <2, 0, 4>, 2 }
   bumpmap_red1
   }
```

The bumpmap is declared using u/v coordinates so that it will follow the natural coordinates of the object. This ensures that it wraps properly around the torus, cone, and sphere in the example above. There is an automatic wrapping of bump maps, so there will be 8 tiles in the u direction and 6 tiles in the v direction of each object.

Sample file:

2.4.3.2 Functional Textures

The most general and flexible texture type is the functional texture. These textures are evaluated at run-time based on the expressions given for the components of the lighting model. The general syntax for a surface using a functional texture is:

```
special surface {
   [ surface declaration ]
   }
```

In addition to the components usually defined in a surface declaration, it is possible to define a function that deflects the normal, and a function that modifies the intersection point prior to texturing. The format of the two declarations are:

```
position vector
normal vector
```

An example of how a functional texture can be defined is:

```
define sin_color_offset (sin(3.14 * fmod(x*y*z,1) + otheta)+1)/2
define sin_color <sin_color_offset, 0, 1 - sin_color_offset>

define xyz_sin_texture
texture {
   special surface {
      color sin_color
      ambient 0.2
      diffuse 0.7
      specular white, 0.2
      microfacet Reitz 10
      }
   }
```

In this example, the color of the surface is defined based on the location of the intersection point using the vector defined as sin_color. Note that sin_color uses yet another definition.

The position declaration is useful to achieve the effect of turbulence. By adding a solid noise to the position, it is possible to add swirl to a basic texture. For example:

```
define white_marble
texture {
   special shiny {
      color white_marble_map[sawtooth(x)]
      position P + dnoise(P, 3)
      }
```

This will create a basic white marble (which would have very straight, even bands of color), and by adding dnoise, swirls the colors around.

The normal declaration is used to add some bumpyness to the surface. The bumps can be created with a statement as simple as:

```
normal N + (dnoise(P) - <0.5, 0.5, 0.5>)
```

The value <0.5,0.5,0.5> is subtracted from the dnoise function since dnoise only returns positive values in each component.

Note: if the color component has any alpha in it (as the result of a lookup from a color map or from an image) then that alpha value will be used as the scale for the transmission statement.

Sample files: cossph.pi, cwheel.pi.

2.4.3.3 Indexed Textures and Texture Maps

A texture map is declared in a manner similar to color maps. There is a list of value pairs and texture pairs, for example:

```
define index_tex_map
   texture_map([-2, 0, red_blue_check, bumpy_green],
               [0, 2, bumpy_green, reflective_blue])
```

Note that for texture maps there is a required comma separating each of the entries.

These texture maps are complimentary to the indexed texture (see below). Two typical uses of indexed textures are to use solid texturing functions to select (and optionally blend) between complete textures rather than just colors, and to use image maps as a way to map textures to a surface.

For example, using the texture map above on a sphere can be accomplished with the following:

```
object {
   sphere <0, 0, 0>, 2
   texture { indexed x, index_tex_map }
   }
```

The indexed texture uses a lookup function (in example above a simple gradient along the x axis) to select from the texture map that follows. See the data file indexed1.pi for the complete example.

As an example of using an image map to place textures on a surface, the following example uses several textures, selected by the color values in an image map. The function indexed_map returns the color index value from a color mapped image (or uses the red channel in a raw image). The example below is equivalent to creating a material map in the POV-Ray raytracer.

```
object {
   sphere <0, 0, 0>, 1
   texture {
      indexed indexed_map(image("txmap.tga"), <x, 0, y>, 1),
              texture_map([1, 1, mirror, mirror],
                          [2, 2, bright_pink, bright_pink],
                          [3, 3, Jade, Jade])
      translate <-0.5, -0.5, 0> // center image
      }
   }
```

In this example, the image is oriented in the x-y plane and centered on the origin. The only difference between a indexed_map and a planar_imagemap is that the first (indexed_map) returns the index of the color in the image and the second returns the color itself. Note that the texture map shown above has holes in it (between the integer values), however this isn't a problem as the indexed_map function will only produce integers.

In addition to the function indexed_map, there is also cylindrical_indexed_map and spherical_indexed_map. These are variations on the indexed_map() that do their thing by wrapping around spherically and cylindrically.

2.4.3.4 Layered Textures

Layered textures allow you to stack multiple textures on top of each other.
If a part of the texture is not completely opaque (non-zero alpha), then the layers below will show through. For example, the following texture creates a texture with a marble outer layer and a mirrored bottom layer and applies it to a sphere:

```
include "colors.inc"

define marble_alpha_map
   color_map([0.0, 0.1, white, 0,   black, 0]
             [0.1, 0.3, black, 0,   white, 0]
             [0.3, 0.6, white, 0,   white, 0.2]
```

```
              [0.6, 1.0, white, 0.2, white, 1])

   define mirror_veined_marble
   texture {
      layered
         texture {
            special shiny { color marble_alpha_map[marble_fn] }
            },
         mirror
      }

   object {
      sphere <0, 0, 0>, 2
      mirror_veined_marble
      }
```

Sample files: Stone1-Stone24.pi, layer1.pi, layer2.pi

2.4.3.5 Summed Textures

Summed textures simply add weighted amounts of a number of textures together to make the final color. The syntax is:

```
   texture {
      summed f1, tex1, f2, tex2, ...
      }
```

The expressions f1, f2, … are numeric expressions. The expressions tex1, … are textures.

Sample file: blobtx.pi

2.5 3D line drawing

It is now possible to draw single pixel wide points and lines.

```
   point location, color

   draw min_t, max_t, steps, location, color
```

For example, to draw a spline curve on the screen you could do the following:

```
   define sp1 [<-1, -1, 0>, <-0.5, 0.2, 0>,
               < 2,  1, 0>, < 1,   1.5, 0>,
               < 2,  2, 0>]
   define sp3 [cyan, blue, magenta]

   draw 0, 1, 256, spline(1, u, sp1), spline(0, u, sp3)
```

This will draw the spline curve sp1 using 256 line segments. The color of the spline will vary from cyan to blue to magenta along the spline.

2.6 Lens flares

Lens flares are an effect that can be added to a textured light (and only to textured lights). The flares are defined with the following format:

```
textured_light {
   color ...
   flare {
      color color_expression
      count num_flares
      spacing power_fn
      seed random_seed
      size min_radius, max_radius
      concave concave_convex_ratio
      sphere glow_radius
      }
   }
```

All values have defaults:

```
color    = White
count    = 10
spacing  = 1.0
seed     = 0
min_rad  = 0.005    // Min size is at least 1/200 of screen size
max_rad  = 0.05     // Max size is at most 1/20 of screen size
concave  = 0.75     // Approximately 3/4's of the rings are concave
sphere   = 0.0      // No glow around the light
```

Definitions of the components of the flare declaration:

The color of the flare can either be solid color or color expression. Color maps can really help the look of a flare.

The value given in spacing determines how bunched the flare rings are around the light source.

The value of count determines how many flare rings will appear

The value of seed is used to change the placement of the flare rings. (They are randomly placed along the line between the light and the center of the screen with the random number generator initialized to the seed value.)

The minimum and maximum radius values determine how large the flare rings will be on the screen.

The value after the sphere declaration determines the size of the flare surrounding the light. By default this is 0. Note that by setting this value and setting count to 0 you will get just a glow around the light.

There are several runtime variables that have special meaning when evaluating the color expression in a lens flare:

```
u - Distance from the center of the flare (at the current pixel)
v - Angle from the x-axis to the current pixel
w - Which flare (between 0 and count-1)
x - 1 if the flare has a hot center, 0 if flare has a hot outer edge
```

For example, the following creates a light with yellowish lens flares:

```
define flare_color01 (gold + white) / 2
define flare_color02 (yellow + white) / 2
define flare_color03 (red + white) / 2
static define flare_map0
   color_map([0.00, 0.20, flare_color01, 0.4, flare_color02, 0.4]
             [0.20, 0.40, flare_color02, 0.4, flare_color03, 0.8]
             [0.40, 1.00, flare_color03, 0.8, black,        1.0])
static define flare_map1
   color_map([0.00, 0.10, black,         1.0, flare_color01, 0.4]
             [0.10, 0.20, flare_color01, 0.4, flare_color02, 0.4]
             [0.20, 0.40, flare_color02, 0.4, flare_color03, 0.8]
             [0.40, 1.00, flare_color03, 0.8, black,        1.0])
textured_light {
   color 0.8*white
   translate light0_loc
   flare {
      color (x > 0 ? flare_map0[u] : flare_map1[u]) // red
      size 0.003, 0.1
      count 15
      seed 2020
      }
   }
object { sphere light0_loc, 0.3 shading_flags 0
         texture { surface { color white ambient 1 diffuse 0 } } }
```

The sphere declaration has shading flags set so that it won’t block the light that is sitting inside. It’s just there so you can see where the light is placed.

2.6.1 Comments

Comments follow the standard C/C++ formats. Multiple line comments are enclosed by /* … */ and single line comments are preceded by //.

Single line comments are allowed and have the following format:

\// [any text to end of the line]

As soon as the two characters // are detected, the rest of the line is considered a comment.

2.7 Animation support

An animation is generated by rendering a series of frames, numbered from 0 to some total value. The declarations in Polyray that support the generation of multiple Targa images are:

```
total_frames val     - The total number of frames in the animation
start_frame val      - The value of the first frame to be rendered
end_frame val        - The last frame to be rendered
frame_time val       - Duration of each frame (for particles)
outfile "name"       - Polyray appends the frame number to 'name'
outfile name           in order to generate distinct Targa files.
```

The values of total_frames, start_frame, and end_frame, as well as the value of the current frame, frame, are usable in arithmetic expressions in the input file. Note that these statements should appear before the use of: total_frames, start_frame, end_frame, or frame as a part of an expression.
Typically declarations are put right at the top of the file.

Sample files: whirl.pi, plane.pi, squish.pi, many others

2.7.1 Tuning an animation

For many animations the actual # of frames is something that you will want to tune. You may want a few frames in trial animations (leading to a jerky playback), with a larger number in the final animation. To do this, you should define variables that turn the total number of frames into a variable with known bounds (e.g., the variable starts at 0.0 on the first frame and ends at 1.0 on the last frame).

The following declaration shows how an animation can be built using this technique. The total number of frames is declared at the top of the file.
Then the variable t is declared which will range from 0 to 1:

```
total_frames 60 start_frame 0 end_frame total_frames

define t frame / (total_frames - 1)
```

Under some circumstances you will will want to have t go from 0 to just under 1. This is particularly useful for a looping animation. For example, if you wanted to rotate an object through 360 degrees, and have the flic play seamlessly, you could use the following declarations: 84

```
total_frames 30 start_frame 0 end_frame total_frames - 1

define t frame / total_frames

…

object { … rotate <0, t*360, 0> }
```

By avoiding the explicit use of ‘frame’ in your data file you can easily modify the smoothness of the final animation. Note that if you are using particle systems you will need to adjust the value of ‘frame_time’ at the same time you modify the value of ‘total_frames’ (see section 2.3.5).

2.8 Conditional processing

In support of animation generation (and also because I sometimes like to toggle attributes of objects), polyray supports limited conditional processing. Note that version 2.0 requires semicolons. The syntax for this is:

```
if (cexper) {
   [object/light/... declarations]
   };
else {
   [other object/light/... declarations]
   };
```

The sample file rsphere.pi shows how it can be used to modify the color characteristics of a moving sphere.

The use of conditional statements is limited to the top level of the data file. You cannot put a conditional within an object or texture declaration. i.e.

```
object {
   if (foo == 42) {
      sphere <0, 0, 0>, 4
      };
```

```
      else {
         disc <0, 0, 0>, <0, 1, 0>, 4
         };
      };
```

is not a valid use of an if statement, whereas:

```
   if (foo == 42) {
      object {
         sphere <0, 0, 0>, 4
         }
      }
   else {
      object {
         disc <0, 0, 0>, <0, 1, 0>, 4
         }
      }
```

or if (foo == 42) object { sphere <0, 0, 0>, 4 } else if (foo = 12) { object { torus 3.0, 1.0, <0, 0, 0>, <0, 1, 0> } object { cylinder <0, −1, 0>, <0, 1, 0>, 0.5 } } else object { disc <0, 0, 0>, <0, 1, 0>, 4 }

are valid.

Note: the curly brackets { } are required if there are multiple statements within the conditional, and not required if there is a single statement.

2.9 Include files

In order to allow breaking an input file into several files (as well as supporting the use of library files), it is possible to direct polyray to process another file. The syntax is:

```
   include "filename"
```

Beware that trying to use #include ... will fail.

Polyray searches for include files in all directories that appear in the environment variable POLYRAY_PATH. This is a big help if you want to keep just a single copy of .inc files and want Polyray to go get them from a specific directory without explicitly giving the path within the data file. (Works just like the DOS PATH statement for executables.)

For example, if you keep your normal colors and texture include files in c:\polyray\include, and keep a bunch of image files for mapping in the directory c:\images then you could set the path in your System Properties. Choose "Environment Variables". And modify or add POLYRAY_PATH to be:

```
   ;c:\polyray\include;c:\images
```

2.10 File flush

This is no longer an option in Polyray. Due to the change to full screen buffering of images, it is no longer possible to flush partial images to disc. If there is a catastrophic failure during rendering (power failure, reset button, ...) you will lose the current image.

2.11 System calls

The system call allows you to invoke an external program from within Polyray. This is useful if you have a program that will generate new data for every frame of an animation. By using system, you can invoke the external program for every frame of the program, and possibly pass it the frame number so that it will know the specific data to generate.

The format of the declaration is:

system(arg1, …, argn)

The arguments are concatenated together (no spaces added) and then passed to the system to execute. Any numeric or string variable is allowed for an argument. The number of arguments isn't limited, however the total number of characters in the final string must be less than 255.

One possible use for the system call is to use DTA to extract a particular frame of an animation for use in an image map. By incrementing the frame number that is retrieved, you can embed an animation into your own animation.

3 File formats

This section describes the file formats used by Polyray. In general there are three types supported: Targa, PNG, and JPG (JFIF format). The only output format currently supported is Targa. All three formats may be used as input images for the following purposes: image maps, bump maps, height fields, depth maps, and gridded objects.

3.1 Output image files

Currently the output file formats are 8, 16, 24, and 32 bit uncompressed and RLE compressed Targa. If no output file is defined with the '-o' command line option, the file out.tga will be used. The command line option -u specifies that no compression should be used on the output file.

The default output format is an RLE compressed 24 bit color format. This format holds 8 bits for each of red, green, and blue. If you don't have the hardware to display 24 bit images (and aren't planning to send them to someone that can), then you can output 16 bit image by using the command line flag

-p 16 (or use 'pixelsize 16' in polyray.ini).

3.2 Input image files

Input image files can be broadly divided into three categories: multichannel (e.g., 24 bit images with 8 bits per pixel), indexed color (e.g., 8 bit index into a palette of 24 bit colors), and greyscale (8 or 16 bits per pixel). How the pixel and palette information is used depends on what the image is being used for. The relevant descriptions for input images are:

image maps 3.2.1 Image maps height fields 3.2.2 Depth map formats depth maps 3.2.2 Depth map formats bump maps 2.4.3.2.3 Bumpmaps gridded objects 2.3.4 Gridded objects

3.2.1 Image maps

planar_imagemap(image, vector [, rflag])

3.2.2 Depth map formats

The interpretation of an image as a depth map depends on the number of bytes per pixel and the type of image file. The next four sections describe how these files are interpreted based on pixel size (and then by image type). Since the use of an image file for storing depth information is not standardized, these are unique to Polyray. (Almost unique, nearly the same interpretation of depth for 8, 16, and 24 bit images have been implemented in POV-Ray by Alexander Enzmann the original developer of Polyray.)

3.2.2.1 8 Bit Format

Each pixel in the file is represented by a single byte. The value of the byte is used as an integer height between -127 and 128. This format includes both indexed color images and greyscale images. The color palette in an indexed image is ignored.

3.2.2.2 16 Bit Format

Each pixel in the file is represented by two bytes, low then high. The high component defines the integer component of the height, the low component holds the fractional part scaled by 255. The entire value is offset by 128 to compensate for the unsigned nature of the storage bytes. As an example the values high = 140, low = 37 would be translated to the height:

```
(140 + 37 / 256) - 128 = 12.144
```

similarly if you are generating a Targa file to use in Polyray, given a height, add 128 to the height, extract the integer component, then extract the scaled fractional component. The following code fragment shows a way of

generating the low and high bytes from a floating point number.

```
unsigned char low, high;
float height;
FILE *height_file;

...

height += 128.0;
high = (unsigned char)height;
height -= (float)high;
low = (unsigned char)(256.0 * height);
fputc(low, height_file);
fputc(high, height_file);
```

This format includes both 16-bit color images and 16 bit greyscale images.

3.2.2.3 24 Bit Format

The red component defines the integer component of the height, the green component holds the fractional part scaled by 255, the blue component is ignored. The entire value is offset by 128 to compensate for the unsigned nature of the RGB values. As an example the values r = 140, g = 37, and b = 0 would be translated to the height:

```
(140 + 37 / 256) - 128 = 12.144
```

similarly if you are generating a Targa file to use in Polyray, given a height, add 128 to the height, extract the integer component, then extract the scaled fractional component. The following code fragment shows a way of generating the RGB components from a floating point number.

```
unsigned char r, g, b;
float height;
FILE *height_file;

...

height += 128.0;
r = (unsigned char)height;
height -= (float)r;
g = (unsigned char)(256.0 * height);
b = 0;
fputc(b, height_file);
fputc(g, height_file);
fputc(r, height_file);
```

3.2.2.4 32 Bit Format

The four bytes of a 32 bit image are used to hold the four bytes of a floating point number. The format of the floating point number is machine specific, and the order of the four bytes in the image correspond exactly to the order of the four bytes of the floating number when it is in memory. This means that if the depth map is created on a computer with a different CPU than the one you are rendering on you may have byte ordering problems.
For modern portability, this old 32-bit depth format is best documented as **legacy/raw native float format**, because it is not guaranteed to be identical across all machines.

For example, the following code shows how to take a floating point number and store it to a file in the format that Polyray will expect. You will also need to write the Targa header, etc.

unsigned char *byteptr; float depth; FILE* ofile;

… calculations for depth …

/* Store a floating point number as a 32 bit color- this is obviously a machine specific result for the floating point number that is stored. There is also an assumption here that a float type is exactly 4 bytes and that the size of an unsigned char is exactly 1 byte. */ byteptr = (unsigned char *)&depth; fputc(byteptr[0], ofile); fputc(byteptr[1], ofile); fputc(byteptr[2], ofile); fputc(byteptr[3], ofile);

This is the modern C++ equivalent example. It writes a 32-bit floating point depth value directly
to a binary output stream. This preserves the historical Polyray behaviour: the bytes are written exactly as the `float` is represented in memory.

#include #include #include #include #include

void write_depth_as_32bit(std::ostream& out, float depth) { static_assert(sizeof(float) == 4, “This format expects a 32-bit float.”); static_assert(sizeof(std::uint8_t) == 1, “This format expects 8-bit bytes.”);

```
const auto bytes = std::bit_cast<std::array<std::uint8_t, 4>>(depth);

out.write(reinterpret_cast<const char*>(bytes.data()), bytes.size());

if (!out) {
    throw std::runtime_error("Failed to write 32-bit depth value.");
}
```

}

```
```

4 Outstanding Issues

Polyray will probably never be done. And in this respect, the next two sections list some things that are planned for the future, as well as things that should already have been fixed.

4.1 To do list

- Improvement of root solving.
- Change to modern C++, improving memory allocation.
- Add defined color_maps to noise surfaces. Currently you have to put the entire color map into the noise surface declaration.
- Parameterised objects and textures. This may occur in the next major release of Polyray, due to the extreme structural changes required.
- Appending .pi to input file names that do not have an extension.
- Trim curves for NURBS
- ‘fat’ 3d lines and points to make better looking particle systems.

4.2 Known Bugs

- It is possible to specify surface components that have no meaning for the type of surface that is being declared (e.g., octaves in a special surface).
- Shading in scan conversion doesn’t always match that from ray-tracing. Not much can be done about this other than increasing the values of u_steps and v_steps. (This needs to be checked in the latest versions) - this may have been fixed in the newer C++ version, will recheck.
- Refraction does not appear to be correct when the ray passes through several overlapping transparent objects. Works fine if the ior is 1.0, or if there is only 1 object intersected by the ray. The cure is to use clipping or u/v bounds to remove the overlapping pieces.
- Currently eval_div in eval.c is performing a calculation, then discarding it by working out the cross product. This is clearly wrong. Documentation does not even mention a vector divided by another vector. This is fixed in the newer C++ version.

- There is an overflow issue in the Bezier Control Points. Fixed in the newer C++ version.

5 Revision History

Version 1.9.4
“Bella” Build (named after Malta’s Eurovision 2026 Entry) Released 16th May 2026 Switched to gcc 16.1.0 on MSYS2. Various fixes to make it more secure. This is the first open source version of Polyray with the blessing of the original developer.

Version 1.9.3
“Travel Malta” Build (available on Github from this version) Released 12th January 2023 Switched to gcc 13.2.0 on MSYS2. Various fixes for old C cruft including uninitalised variable, fixing prototypes. Changes for Linux and Macos compatilbity (Mac version will be available in the future). This is still based on the stable C code but a newer C++ version is worked on (not directly based on version 1.9.2 beta). Updated parser grammar. Skipcom has been deprecated and isn’t used. File includes work differently and environment variables should now work better under Windows.

Version 1.9.2
“Second Life” Beta Build Released: 31st January 2020 With full installer. Switched to gcc 9.2.0, Mingw64/Msys2.0. Further code changes. Switched to stb library for image loading, removed the old code for tga/jpeg reading. Exper_node is now a std::variant. Introduced the use of std::arrays. Img is now cImg, a C++ class. Bug fix in polynomial roots algorithm. Reduction of old C cruft and bring it up to date. Code compiles under Visual Studio C++ too. There may be a slight speedup as a result of various changes.

Version 1.9.2
“New Year 2020” Beta Build Released: 5th January 2020 With full installer. Switched to gcc 9.1.0, Mingw64/Msys2.0. Further changes to switch part of the code to C++ standard. There may be a slight speedup as a result.

Version 1.9.1

“Chameleon Malta” Stable Build on occasion of Eurovision Semifinal Released: 16th May 2019 With a full installer Switched to gcc 8.3.0 and Mingw64. Removed some insecure code

Version 1.9f

“Graphics Stars” Build Released: 18th February 2015 With a full installer Recompiled with TDM gcc 4.5.1 - 32 bit. Minor fixes, update to NSIS installer which now sets up Textpad shortcuts for Polyray, calling a Java compiler and the GCC c compiler.

Version 1.9 2009 ‘What if we’ Build.

Full installer. Fixed uv_bounds bug.

Version 1.8e

“Human” Build Released: 23rd February 2011 With a full installer

Version 1.8d

“Paloma Blanca” Build Released: 16th February 2008 With a full installer Recompiled with gcc 4.2.3 unofficial for MingW, with latest optimisations. Added NSIS installer. Minor code fixes.

Version 1.8d 2007 ‘The Core’ Build:

New colours - besides gray, now there is Gray05, Gray10, up to Gray95. SkyBlue1 to 5 (use the new colors.inc in this zip file) New random number generator New matrix multiplication algorithm (faster) Rough speed improvement should be around 33% Fixed problem with some XP PC’s environment variables Compiled with gcc - special mingw version 3.4.5 - updated MingW environment

Set an environment variable POLYRAY_PATH to where you have placed polyray’s dat subdirectory eg c: (under XP do this in the control panel, system) Extended memory or DOS4GW is NOT needed for this version. This is a full win32 mingw gcc compile for the first time.

(Other versions were released to CIS students at the University of Malta but are not listed here)

Version 1.8

Released: 26 October 1995 (last version by the original author)

- Modified the way opacity in a color map interacts with transparency. It is now much closer to the way POV-Ray does it (this made the interface with MORAY far better, even though it kinda breaks the look

of existing files).

- Removed GIF and added PNG image file formats.

- Added cylindrical and spherical indexed maps.

- Added 64K and 16 color video modes

- Keyword "noshadow" can be added to light. Lights may also now be placed into object wrappers for use in CSG objects.

- Added several new primitives: superquadric, spherical height field, cylindrical height field, and an experimental hypertexture primitive.

- Added 3D point and line drawing

- Added several spline types

- added "brilliance" to surface components (power modifier of diffuse component)

- Added simple lens flares

- A full screen z-buffer is now used for all rendering modes. This means you may need to reduce the maximum amount of RAM dedicated to the screen to render large images (see -M command line flag).

- The environment variable POLYRAY_PATH will be used to find any include files. Multiple directories can be put on this variable, in the same way they are for the standard PATH.

Version 1.7

Released: 17 March 1994

- Added parametric surfaces. Now possible to define a mesh object (surface) where each point in the mesh is a function of u and v.

- Added bump maps

- Added noeval to definitions to cure a particle system bug

- Added support for GIF and JPEG images in textures, etc.

- Added support for system calls in 386 version. Can now call an external program from within Polyray.

o The default shading flags for raytracing is now set to:
  SHADOW_CHECK + REFLECT_CHECK + TRANSMIT_CHECK +
  UV_CHECK + CAST_SHADOW
  The difference is that it is no longer assumed that surfaces should be lit on both sides (normal correction), this boosts rendering speed at the cost of funny shading once in a while. UV_CHECK was added to allow turning off checking of u/v bounds on objects (also a speedup).

o Changed u_steps, v_steps back to the way it was in v1.5. Now for all objects it is uniformly subdivided by exactly the value of u_steps/v_steps.

o Significantly enhanced the function object. Polyray will now accept any predefined function as part of the function definition.

o Added NURBS objects. (Sorry, no trim curves yet.)

o Displacement functions are now possible on CSG objects. They also work in raytracing now (although you may need to really boost the u_steps and v_steps values to get good results).

o Now supports /* ... */ comments (C style). No you can't nest them. You can nest the single line // comments within them.

o Internal modifications made that should result in less memory usage by objects. Your mileage may vary.

o Modified numeric input to allow numbers like 1., .1

o Fixed up bug in polygon tracing that caused them to drop out if they were oriented in just the right way.

o Added a second raw triangle output format - only outputs the vertex coordinates. This makes it more compatible with RAW2POV. Previous style with normals and u/v still available.

o Removed BSP tree bounding. It wasn't any faster than slabs and locked up every once in a while.

o Added a glyph object to support TrueType style extruded surfaces. Glyphs are always closed at the top and bottom, they may have both straight and curved sides in the same shape. A program to extract TrueType information and write into Polyray glyph format is included with the data file archive.

o Support for simple particle systems. Can define birth and death conditions, # of objects to generate, and ability to

bounce off other objects.

- Added many more VESA display modes. There are now 5 SVGA 256 color modes, 5 Hicolor modes, and 5 truecolor modes supported. If your board doesn't support a selected mode, Polyray tries a lower res one having the same # of bytes per pixel.

Version 1.6a (Crud, stuff was still broken…)

Released: 23 April 1993

- Problems in writing Targa images! Uncompressed/RLE was being set improperly in the image file.

- Allocation/Deallocation of static variables had some problems. Mostly fixed, however: DONT use a non-static texture in a static object.

- Added some code to special surface to make use of an alpha value in a color map. If you use, color xxx_map[foo_fn], it will grab the alpha value and use it for the transmission scale. (This overrides any transmission scale you might put in later, so beware.)

Version 1.6 (beta)

Released:

- Added opacity values to 16 and 32 bit Targa output. Just a single bit in the 16 bit files. In the 32 bit files the extra channel holds the percentage of the background that contributed to the pixel.

- Added static (last from frame to frame) variables

- Added indexed and summed textures.

- Added RLE compressed 8 bit Targa files.

- Added VESA Hicolor display in 640x480 (still a bit buggy). Don't use anything but '-t 0' or '-t 1' for status displays or the screen will go wierd after about 50 lines.

- Added texture maps (similar to color maps), indexed image files (for use with texture maps)

- Added output of raw triangle information. Render type 3 will render the image as triangles, in ASCII form. Each line of the output has the form: v1 v2 v3 n1 n2 n3 uv1 uv2 uv3. Where vx is a 3D vertex, nx is the normal at the corresponding vertex, and uv1 is a pair of u,v values for each vertex. (This means there are 24 floating point values per line.)

- Fixed bug involving conditional include files. If an include directive was inside a conditional, the file was read regardless of the value of the conditional.

- Smoothed out Bezier surfaces. A patch (smooth) triangle is now used instead of a flat one when performing final intersection tests.

- Fixed view bug in scan conversion - if the direction of view was along the y-axis & the up was along the z-axis, the image came out upside down.

- Added displacement to scan converted surfaces

- Added u-v mapping to most primitives. The variables u and v work in all expressions. This is especially useful for image mapping.

- Added u-v limits to several primitives. This allows for creation of sections of a primitive.

- Changed scan conversion to be adaptive to the on-screen pixel size. Unless limited by u_steps or v_steps, the subdivision of a primitive continues until a polygon about the size of a pixel is created. This is then drawn. The biggest drawback is the appearance of cracks when the adaptive subdivision is different for adjacent parts of a surface.

- Changed the meaning of u_steps and v_steps for most primitives. They now mean the number of binary subdivisions that are performed rather than the number of steps. Therefore u_steps 4 in this version is equivalent to u_steps 16 in previous versions. (Unaffected primitives: blobs, sweeps, lathes. These still work the old way.)

- Fixed dot product of vectors in expressions.

- Added Legendre polynomials as an expression. Pretty cool for doing spherical harmonics.

- Added depth mapped lights. Useful if you need to do shadows of things that can only be scan converted (like displacement surfaces).

- Fixed problems with using a torus in CSG. (Only appeared in some versions of v1.5)

Version 1.5

(not released)

- Buggy SVGA support removed. Only standard VGA mode (320x200) supported.

- Gridded objects added.

- Arrays added

- Components of CSG objects are now properly sorted by bounding slabs

- User defined bounding slabs removed. Polyray will always use bounding slabs aligned with the x, y, and z-axes.

- Clipping and bounding objects removed. Clipping is now performed in CSG, bounding is specified using a bounding_box declaration.

- Added wireframe display mode.

- Added planar blob types. (Also added toroidal blob types, but they only appear in scan conversion images due to the extreme numerical precision needed to raytrace them.)

- Added smooth height fields.

- Fixed shading bug involving transparent objects & multiple light sources.

- Fixed diffuse lighting from colored lights.

- Changed RLE Targa output so that line boundaries are not crossed.

Version 1.4

Released: 11 April 1992

- Support for many SVGA boards at 640x480 resolution in 256 colors. See documentation for the -V flag. (Note: SVGA displays only work on the 286 versions.)

- Changed the way the status output is managed. Now requires a number following the -t flag. Note that line and pixel status will screw up SVGA displays - drawing goes to the wrong place starting around line 100. If using SVGA display then either use no status, or totals.

o Added cylindrical blob components. Changed the syntax for blobs to accommodate the new type.

o Added lathe surfaces made from either line segments or quadratic splines.

o Added sweep surfaces made from quadratic splines.

o Height field syntax changed slightly. Non-square height fields now handled correctly.

o Added adaptive antialiasing.

o Squashed bug in shading routines that affected almost all primitives. This bug was most noticeable for objects that were scaled using different values for x, y, and z.

o Added transparency values to color maps.

o Added new keywords to the file polyray.ini: shadow_tolerance, antialias, alias_threshold, max_samples. Lines that begin with // in polyray.ini are now treated as comments.

o Short document called texture.txt is now included in PLYDOC. This describes in a little more detail how to go about developing solid textures using Polyray.

o Added command line argument -z start_line. This allows the user to start a trace somewhere in the middle of an image. Note that an image that was started this way cannot be correctly resumed & completed. (You may be able to use image cut and paste utilities though.)

Version 1.3

(not released)

o Added support for scan converting implicit functions and polynomial surfaces using the marching cubes algorithm. This technique can be slow, and is restricted to objects that have user defined bounding shapes, but now Polyray is able to scan convert any primitive.

o A global shading flag has been added in order to selectively turn on/off some of the more time consuming shading options. This option will also allow for the use of raytracing as a way of determining shadows, reflectivity, and transparency during scan conversion.

o Added new keywords to the file polyray.ini: pixel_size,

pixel_encoding, shade_flags.

- Improved refraction code to (mostly) handle transparent surfaces that are defined by CSG intersection.

- Fixed discoloring of shadows that receive some light through a transparent object.

- Jittered antialiasing was not being called when the option was selected, this has been fixed.

- Fixed parsing of blobs and polygons that had large numbers of entries. Previously the parser would fail after 50-60 elements in a blob and the same number of vertices of a polygon.

- In keeping with the format used by POV-Ray and Vivid, comments may now start with // as well as #. The use of the pound symbol for comments may be phased out in future versions.

Version 1.2

Released: 16 February 1992

- Scan conversion of many primitives, using Z-Buffer techniques.

- New primitives: sweep surface, torus

- Support for the standard 320x200 VGA display in 256 colors.

- An initialization file, polyray.ini, is read before processing. This allows greater flexibility in tuning many of the default values used by Polyray.

- User defined bounding slabs added. This greatly improves speed of rendering on data files with many small objects.

- Noise surface added.

- Symbol table routines completely reworked. Improved speed for data files containing many definitions.

- Bug in the texturing of height fields corrected.

Version 1.1

(not released)

- Added parabola primitive

- Dithering of rays, and objects

- Blob code improved, shading corrected, intersection code is faster and returns fewer incorrect results.

Version 1.0

Released: 27 December 1991

- Several changes in input syntax were made, the most notable result being that commas are required in many more places. The reason for this is that due to the very flexible nature of expressions possible, a certain amount of syntactic sugar is required to remove ambiguities from the parser.

- Several new primitives were added: boxes, cones, cylinders, discs, height fields, and Bezier patches.

- A new way of doing textures was added - each component of the lighting model can be specified by an implicit function that is evaluated at run time. Using this feature leads to slower textures, however because the textures are defined in the data file instead of within Polyray, development of mathematical texturing can be developed without making alterations to Polyray.

- File flush commands in the data file and at the command line were added.

- Several new Targa variants were added.

- Image mapping added.

- Numerous bug fixes have occurred.

Version 0.3 (beta)

Released: 14 October 1991

- This release added Constructive Solid Geometry, functional surfaces defined in terms of transcendental functions, a checker texture, and compressed Targa output.

- Polyray no longer accepted a list of bounding/clipping objects, only a single object is allowed. since CSG can be used to define complex shapes, this is not a limitation, and even better makes for cleaner data files.

Version 0.2 (beta)

(not released)

- This release added animation support, defined objects, arithmetic expression parsing, and blobs.

Version 0.1 (beta)

(not released)

- First incarnation of Polyray. This version had code for polynomial equations and some of the basic surface types contained in mtv.